Data Structure and Algorithm Design

ME/MSc (Computer) – Pokhara University

Prepared by:

Assoc. Prof. Madan Kadariya (NCIT)

Contact: madan.kadariya@ncit.edu.np

Chapter 5:
Advance Sorting Algorithms (6 hrs)

Outline

5. **Advanced Sorting Algorithms (6 hrs)**
   - 5.1. Non-comparison-based sorting: Radix sort, bucket sort
   - 5.2. Parallel Sorting Algorithms (multi-core and distributed systems)
   - 5.3. External sorting and its applications in large-scale data engineering

Comparison based Sorting (Review)

- A comparison sort is a type of sorting algorithm that only reads the list elements through a single comparison operation (often a "less than or equal to" operator or a three-way comparison) and determines which of two elements should occur first in the final sorted list.
- What is common to all these algorithms?
  - Make comparisons between input elements like:

    $a_i < a_j$, $a_i \leqslant a_j$, $a_i == a_j$, $a_i \geq a_j$, $a_i > a_j$
- Examples of Comparison based sorting
  - Bubble sort
  - Insertion sort
  - Selection sort
  - Shell Sort
  - Quick sort
  - Heap sort
  - Merge sort
  - Etc

Comparison based Sorting (Review)

Complexities

Sorting Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Odd-Even Sort	O(n)	O(n²)	O(n²)	O(1)
Cocktail Sort	O(n)	O(n²)	O(n²)	O(1)
Cycle Sort	O(n²)	O(n²)	O(n²)	O(1)
Merge Insertion (Ford-Johnson) Sort	O(n)	O(n²)	O(n²)	O(n)
Smoothsort	O(n)	O(n log n)	O(n log n)	O(1)
Timsort	O(n)	O(n log n)	O(n log n)	O(n)

Limitations of comparison based Sorting

1. **Lower Bound on Time Complexity**
   - The comparison-based sorting algorithms cannot achieve better than ***O(nlogn)*** time complexity in the **average** and **worst** cases due to the decision tree model, which establishes a lower bound of ***Ω(nlogn)*** for the number of comparisons required.
   - Sorting is like navigating a decision tree***(next slide)***; element comparisons guide branching, and the tree's depth (related to comparisons) determines sorting efficiency, yielding an ***O(nlogn)*** lower bound.
2. **Inefficiency on Large Datasets**
   - For very large datasets, comparison-based sorts can become inefficient in both time and space, especially when simpler algorithms like Counting Sort or Radix Sort can achieve linear time complexity.
   - Even though Quick Sort, Merge Sort, and Heap Sort have ***O(nlogn)*** time complexity, their constant factors and memory usage can still make them slower than non-comparison-based algorithms in certain scenarios.

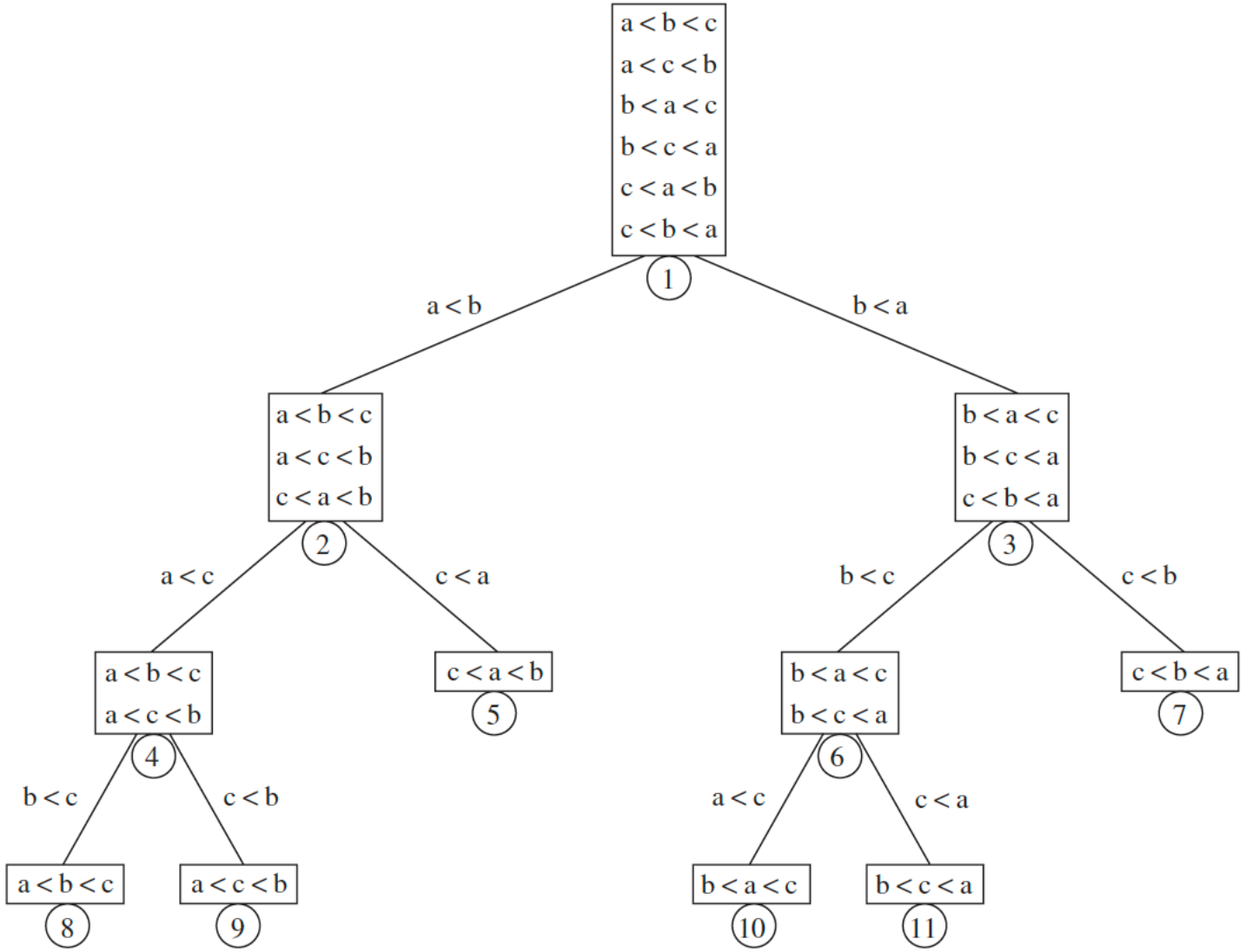


Fig : A decision tree for three-element sort

Limitations of comparison based Sorting

3. **Inability to Exploit Specific Data Properties**
   - Comparison-based sorts do not take advantage of the specific structure of the data being sorted, such as known ranges of integers or fixed alphabets of strings.
   - Since they rely only on comparisons between elements, they cannot use extra information about the data.
4. **Limited Space Efficiency**
   - Some comparison-based sorting algorithms, like Merge Sort, require $O(n)$ extra space for merging, which can be a significant drawback when dealing with large datasets or limited memory.
   - Unlike in-place algorithms such as Quick Sort or Heap Sort, these algorithms are less space-efficient, making them less suit
5. **No Linear-Time Sorting for Certain Data Types**
   - Comparison-based algorithms do not provide **linear time** sorting for certain types of data, such as integers or strings with a limited range of values.
   - Algorithms like **Counting Sort**, **Radix Sort**, and **Bucket Sort** exploit the structure of the data to achieve linear time complexity.

Non-comparison based Sorting

- There are some sorting algorithms that perform sorting without comparing the elements rather by making certain assumptions about the data. They are called the non comparison sorting.
- Non-comparison-based sorting algorithms sort data without comparing elements to each other. Instead, they use properties of the data itself, like digit values or frequency, to determine the order.
- Non comparison sorting include:
  1. **Counting sort** (indexes using key values)
  2. **Radix sort** (examines individual bits of keys)
  3. **Bucket sort** (examines bits of keys)
     - These are Linear sorting algorithms.
     - They make certain assumptions about the data. These type of sorting algorithm does not need to go through the comparison decision tree.

Counting Sort

Counting Sort

- Counting sort is a non-comparison-based sorting algorithm that sorts elements by counting the occurrences of each distinct element and using these counts to determine the correct position of each element in the sorted output. It's efficient for sorting integers within a known range.
- Counting sort assumes that each of the ***n*** input elements is an integer in the range ***0*** to ***k***. that is ***n*** is the number of elements and ***k*** is the highest value element.
- Consider the input set : *4, 1, 3, 4, 3*. Then $n=5$ and $k=4$
- Counting sort determines for each input element x, the number of elements less than x. And it uses this information to place element x directly into its position in the output array. For example if there exists 17 elements less that x then x is placed into the 18th position into the output array.
- The algorithm uses three array:
  - **Input Array**: **A[1..n]** store input data where $A[i] \in \{1, 2, 3, .., k\}$
  - **Output Array**: **B[1..n]** finally store the sorted data ,
  - **Temporary Array**: **C[1..k]** store data temporarily.

Counting Sort (Example)

Counting Sort

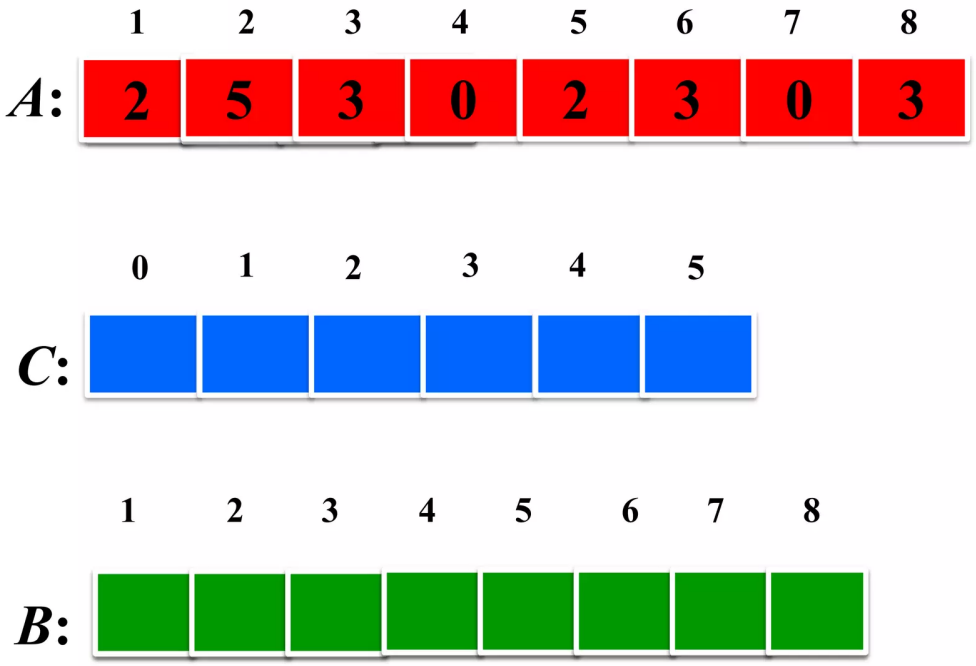
```
1.  Counting-Sort(A, B, k)
2.  Let C[0…..k] be a new array
3.  for i=0 to k
4.      C[i]= 0;
5.  for j=1 to A.length  or n
6.      C[ A[j] ] = C[ A[j] ]  + 1;
7.  for i=1 to k
8.      C[i] = C[i] + C[i-1];
9.  for j=n or A.length down to 1
10.     B[  C[ A[j] ]  ] = A[j];
11.     C[ A[j] ] = C[ A[j] ]  - 1;
```

Counting Sort

```
1.  Counting-Sort(A, B, k)
2.  Let C[0…..k] be a new array
3.  for i=0 to k                          [Loop 1]
4.      C[i]= 0;
5.  for j=1 to A.length  or n             [Loop 2]
6.      C[ A[j] ] = C[ A[j] ]  + 1;
7.  for i=1 to k                          [Loop 3]
8.      C[i] = C[i] + C[i-1];
9.  for j=n or A.length down to 1         [Loop 4]
10.     B[  C[ A[j] ]  ] = A[j];
11.     C[ A[j] ] = C[ A[j] ]  - 1;
```

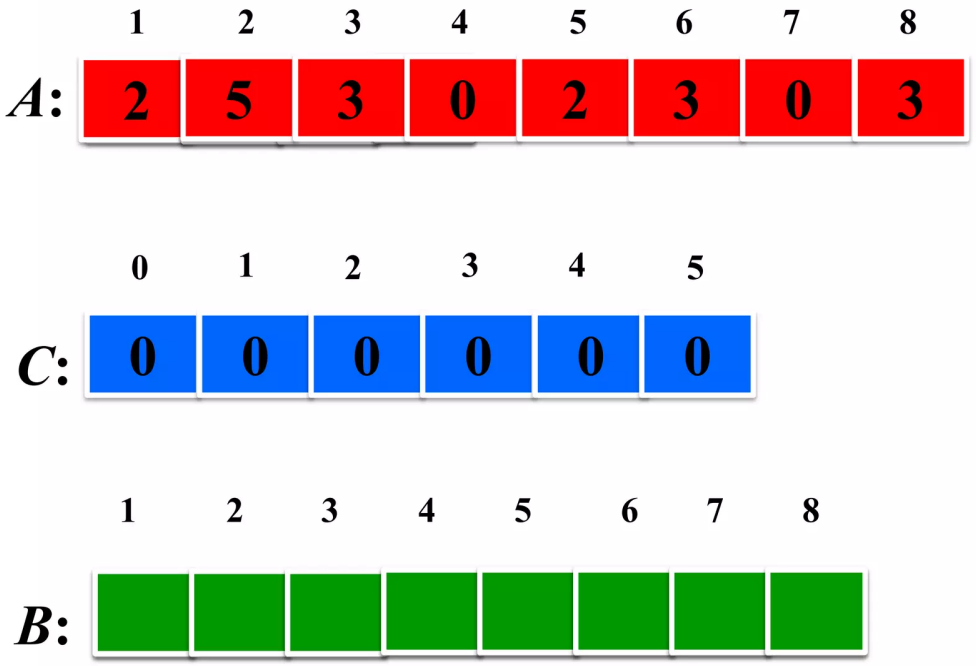
Counting Sort (Example)

Counting-sort example



Executing Loop 1

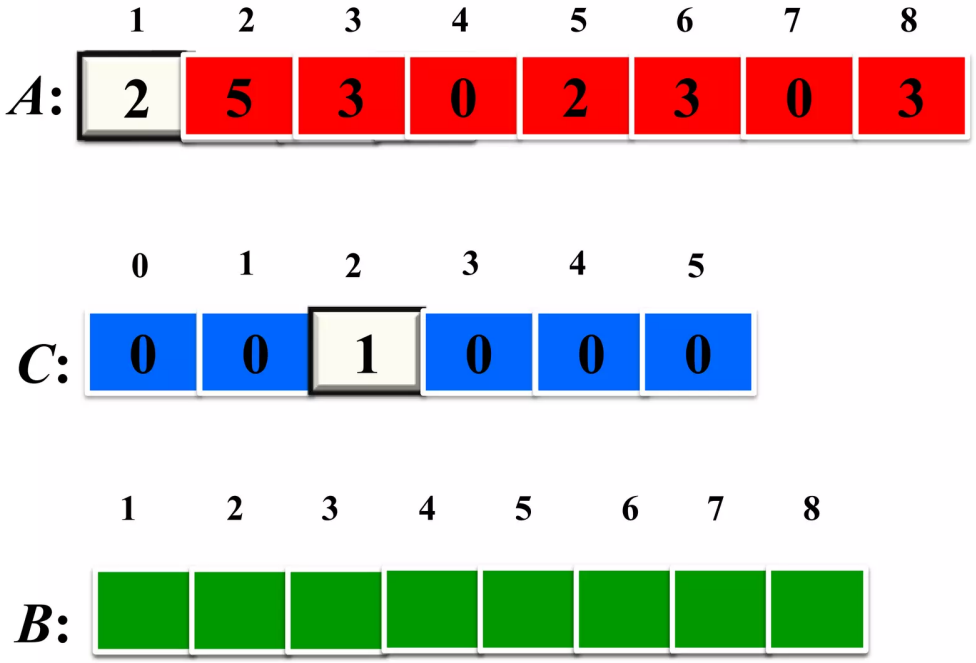
```
3.  for i=0 to k
4.      C[i]= 0;
```



Counting Sort (Example)

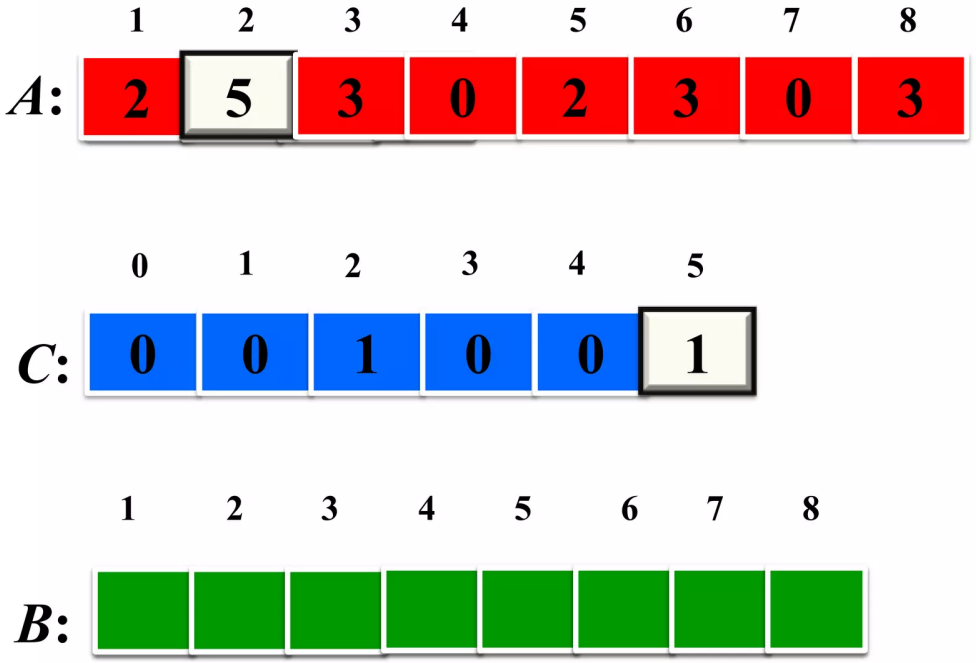
Executing Loop 2

```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



Executing Loop 2

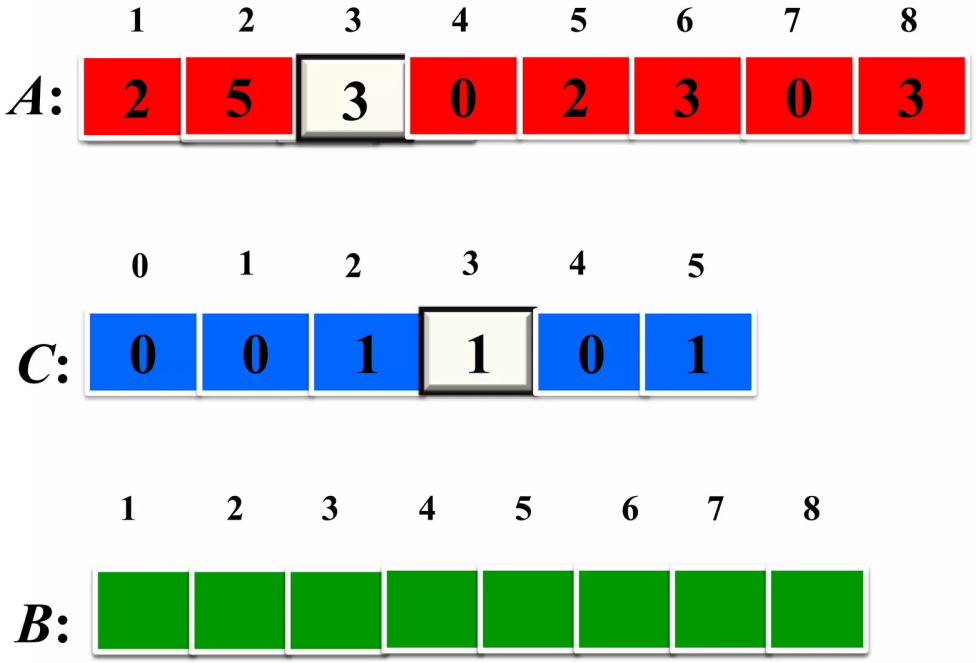
```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



Counting Sort (Example)

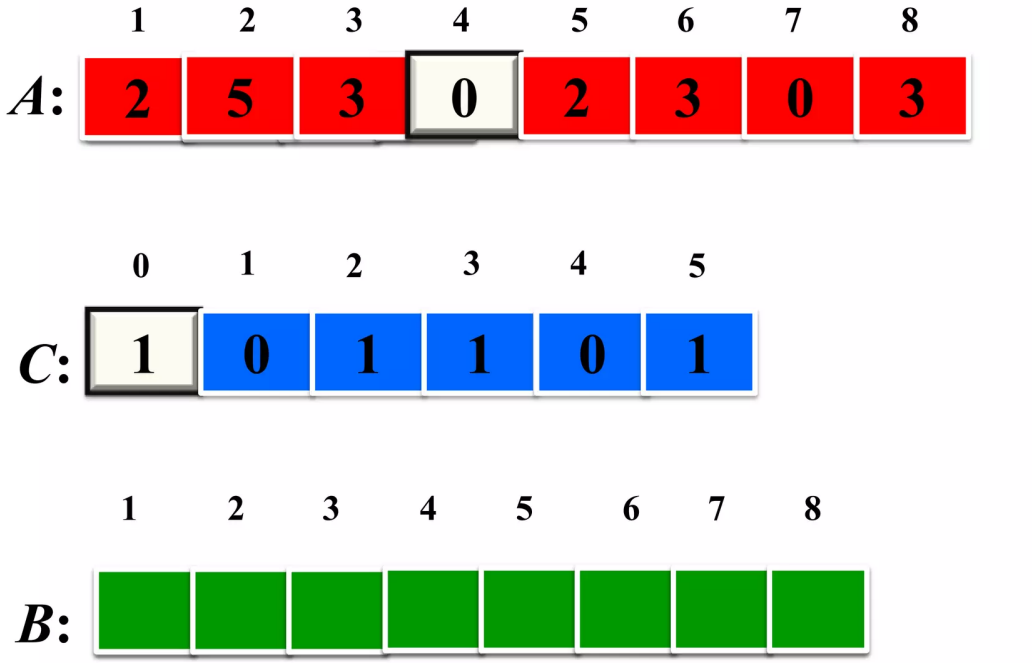
Executing Loop 2

```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



Executing Loop 2

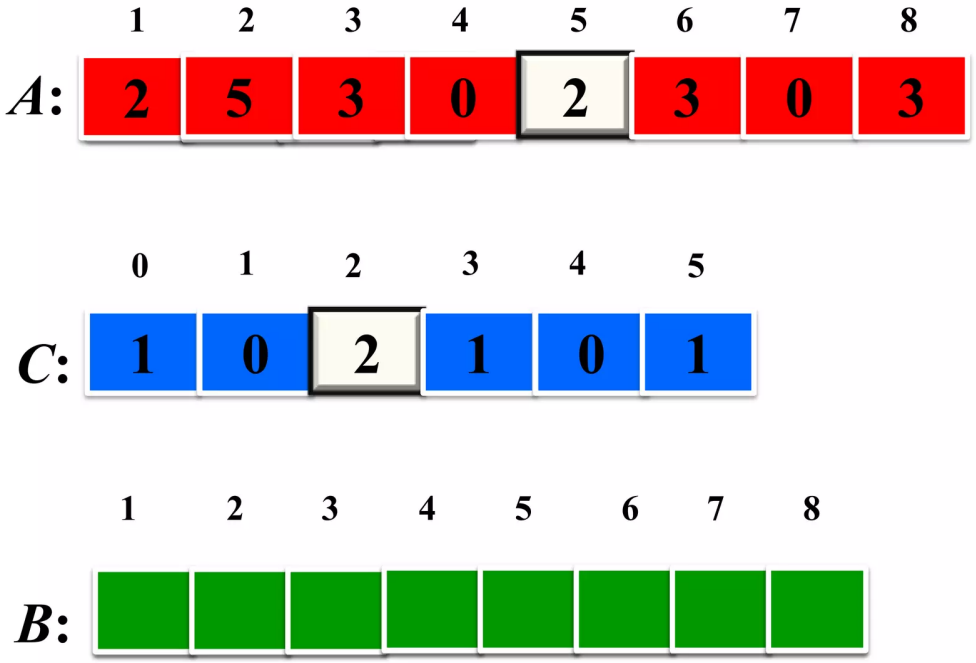
```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



Counting Sort (Example)

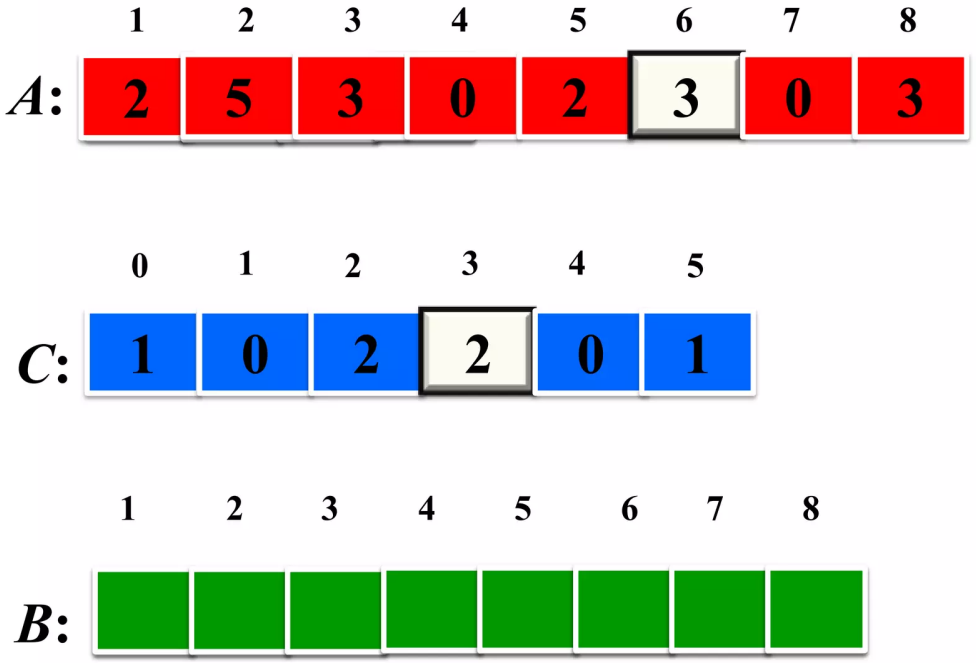
Executing Loop 2

```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



Executing Loop 2

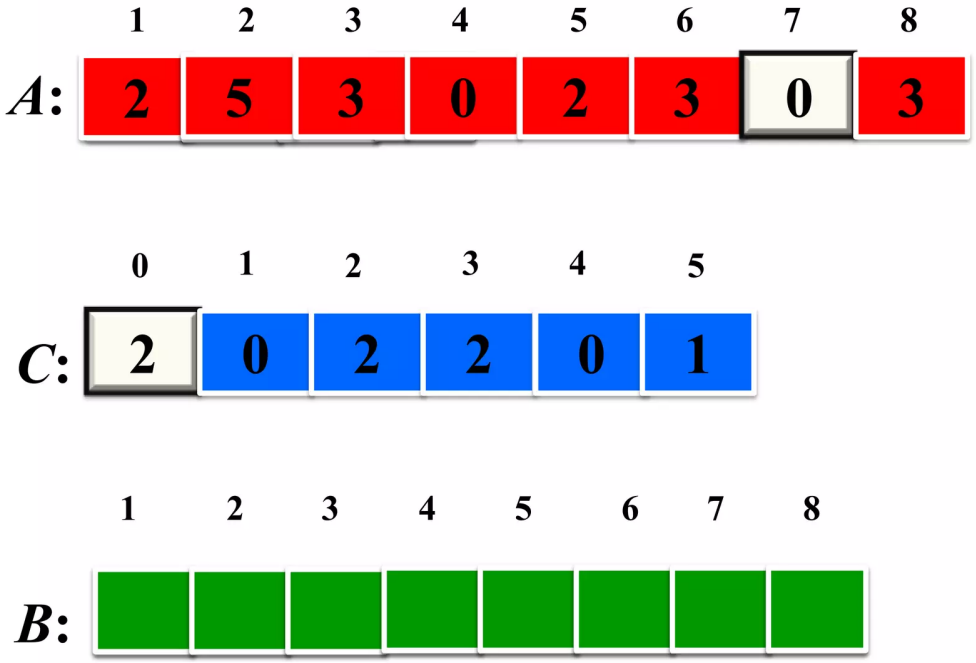
```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



Counting Sort (Example)

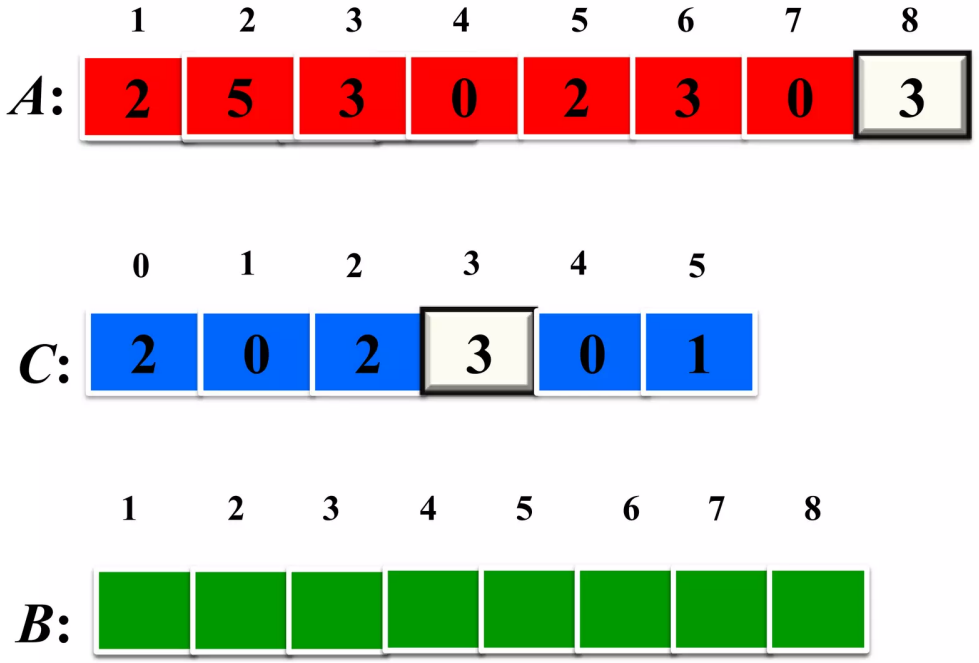
Executing Loop 2

```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



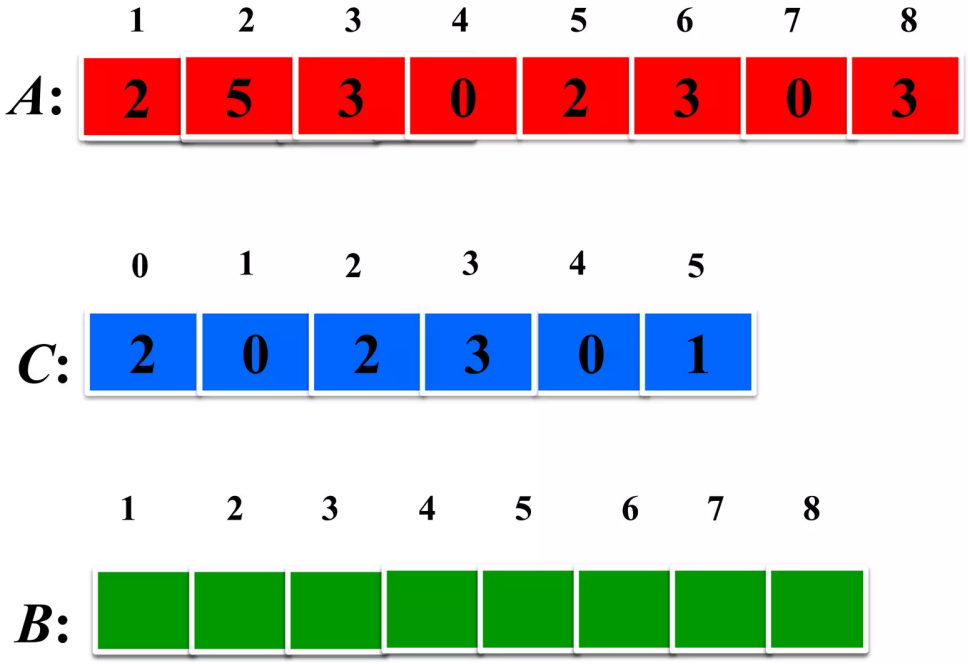
Executing Loop 2

```
5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;
```



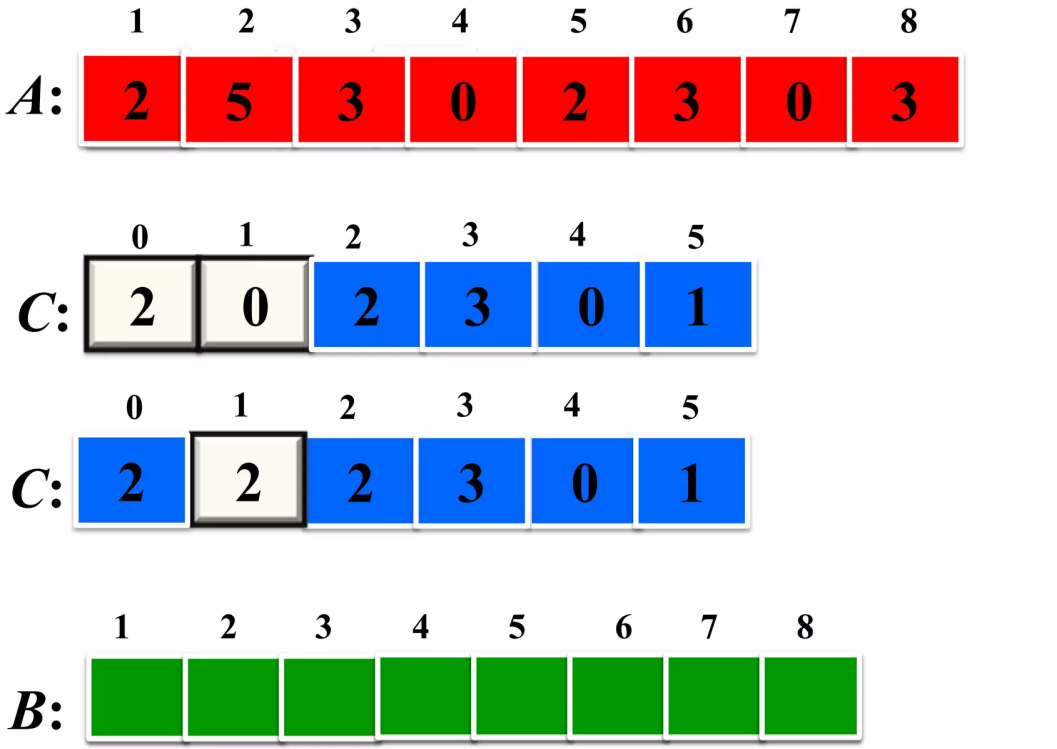
Counting Sort (Example)

End of Loop 2



Executing Loop 3

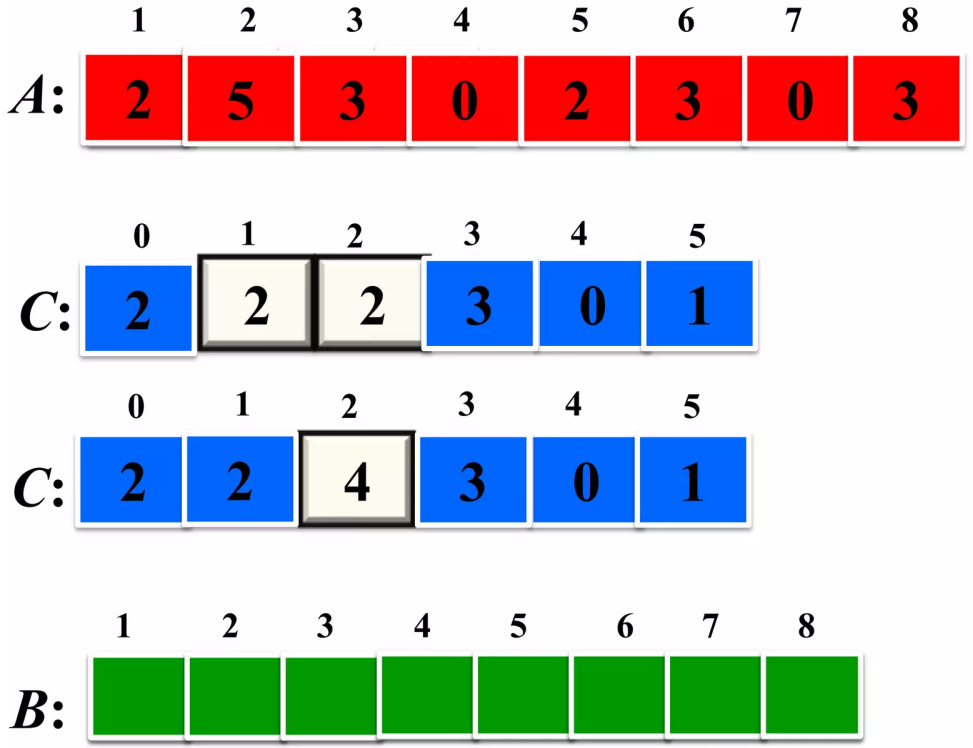
```
7.  for i=1 to k
8.      C[i] = C[i] + C[i-1];
```



Counting Sort (Example)

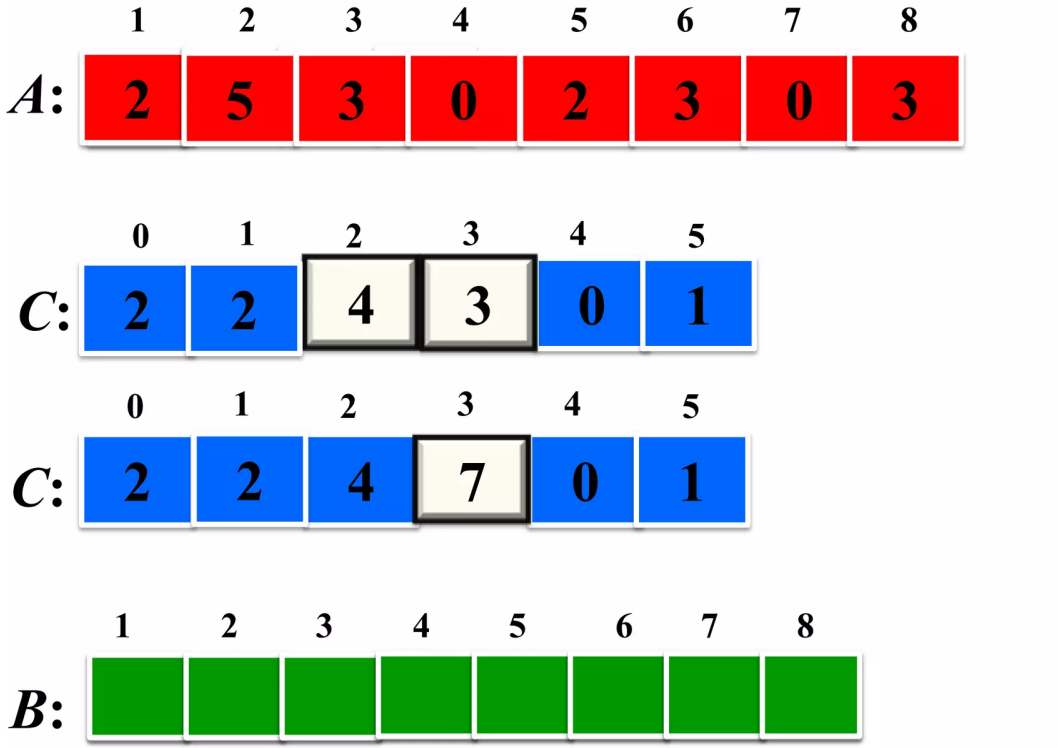
Executing Loop 3

```
7.  for i=1 to k
8.      C[i] = C[i] + C[i-1];
```



Executing Loop 3

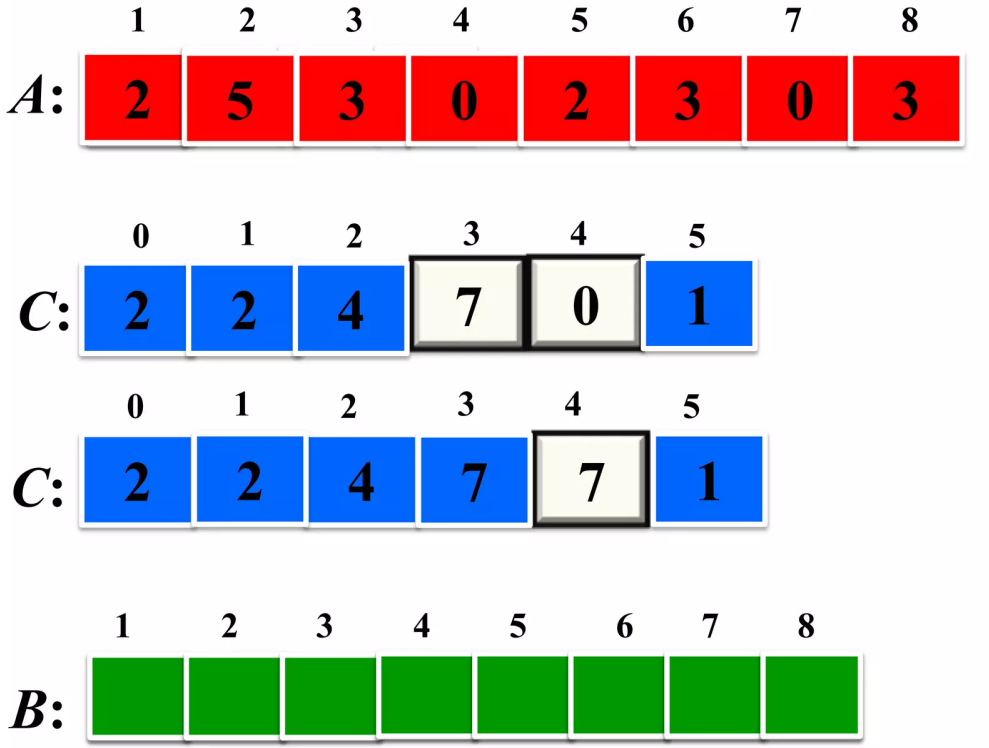
```
7.  for i=1 to k
8.      C[i] = C[i] + C[i-1];
```



Counting Sort (Example)

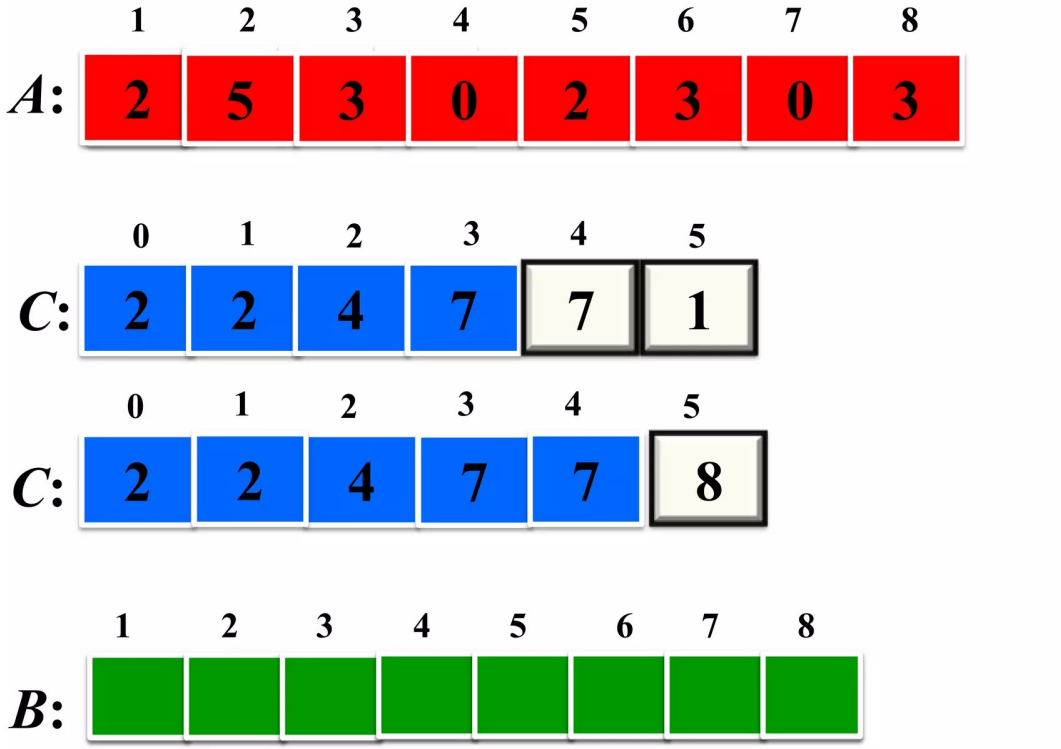
Executing Loop 3

```
7.  for i=1 to k
8.      C[i] = C[i] + C[i-1];
```



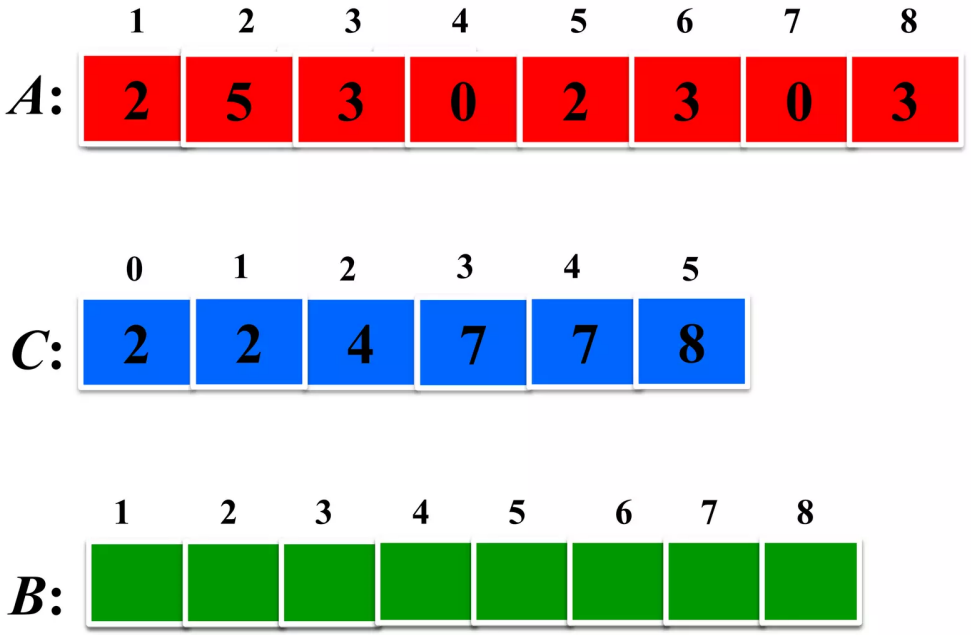
Executing Loop 3

```
7.  for i=1 to k
8.      C[i] = C[i] + C[i-1];
```

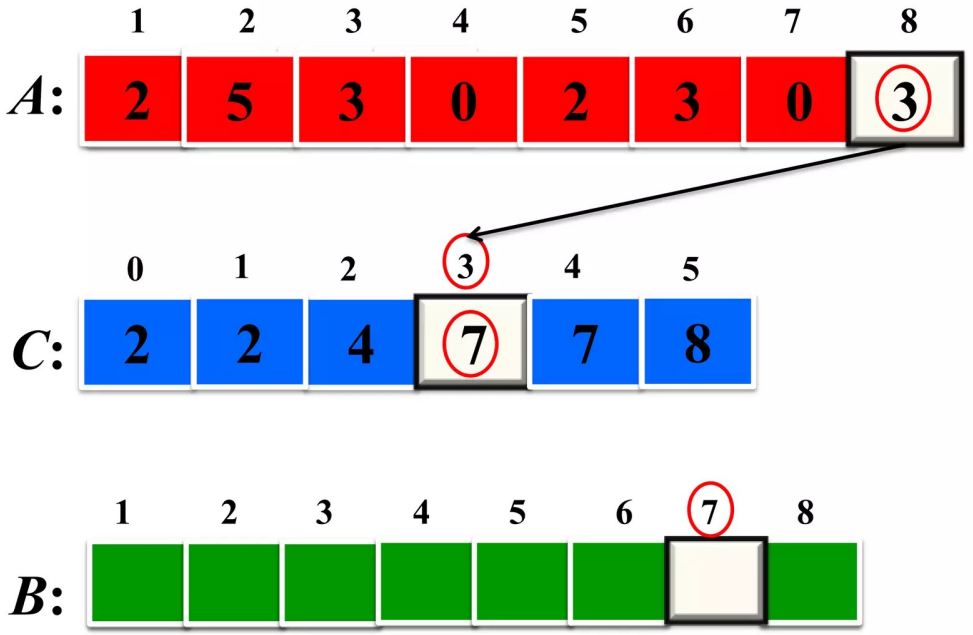


Counting Sort (Example)

End of Loop 3



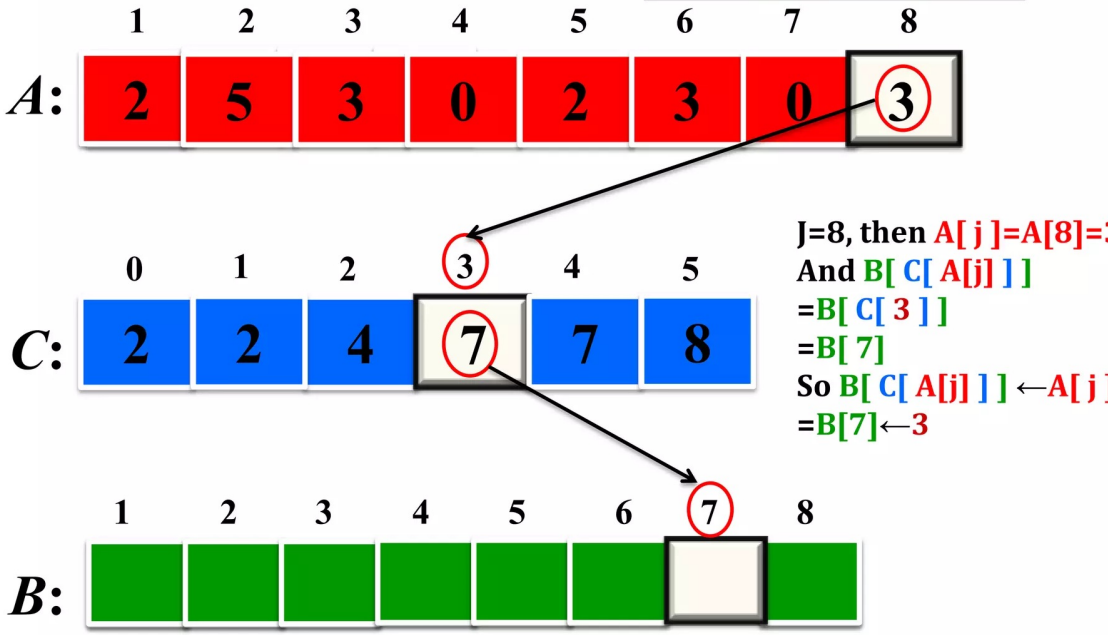
Executing Loop 4



Counting Sort (Example)

Executing Loop 4

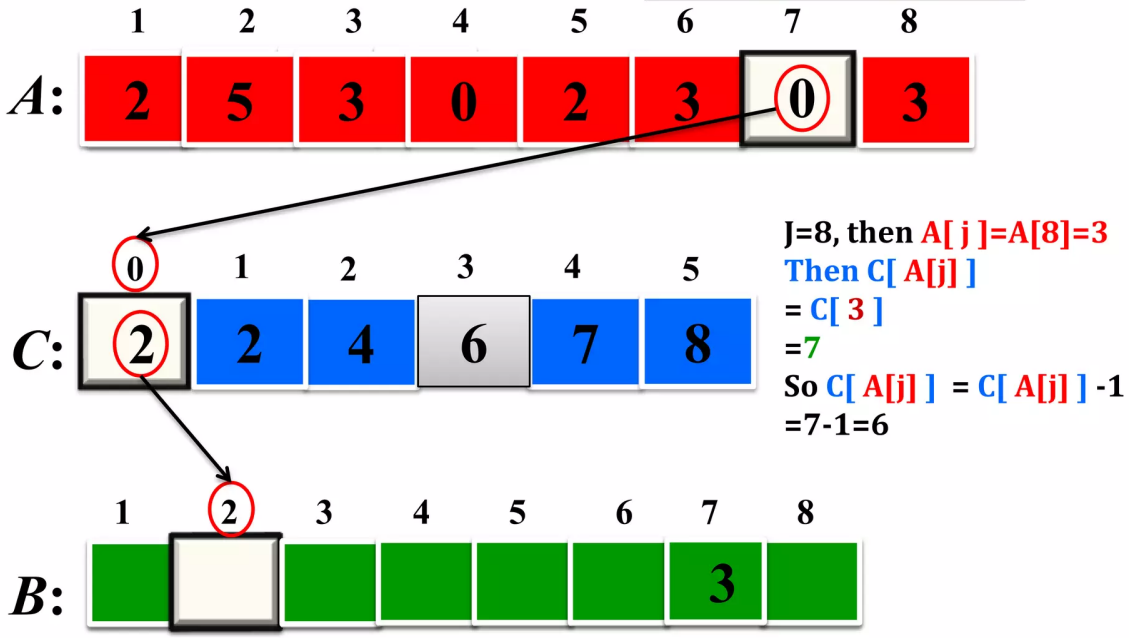
```
9.  for j=n or A.length down to 1
10.     B[ C[ A[j] ] ] = A[j];
11.     C[ A[j] ] = C[ A[j] ] - 1;
```



J=8, then A[j]=A[8]=3
And B[C[A[j]]]
=B[C[3]]
=B[7]
So B[C[A[j]]] ←A[j]
=B[7]←3

Executing Loop 4

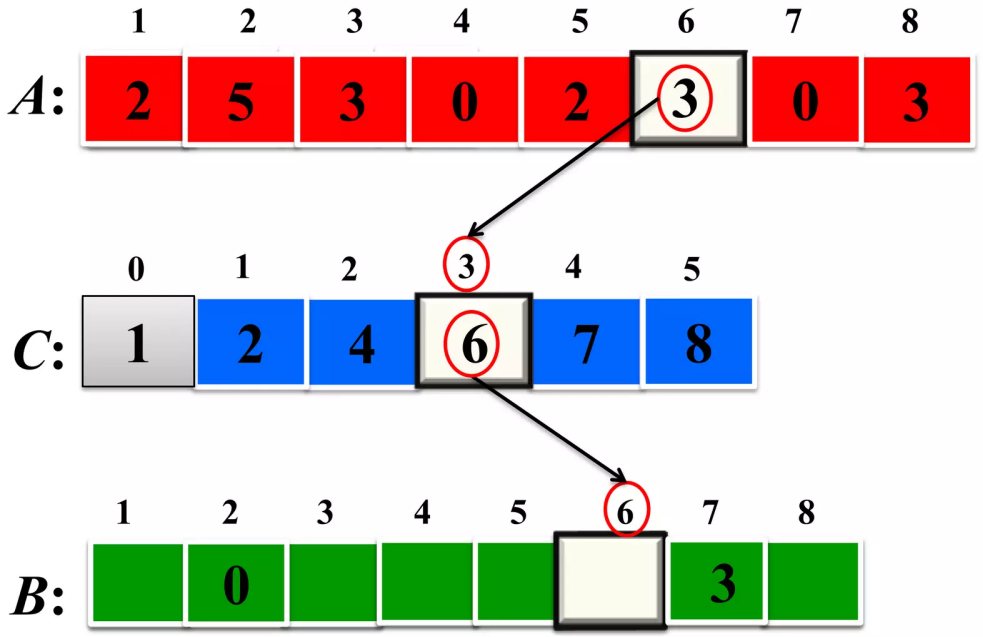
```
9.  for j=n or A.length down to 1
10.     B[ C[ A[j] ] ] = A[j];
11.     C[ A[j] ] = C[ A[j] ] - 1;
```



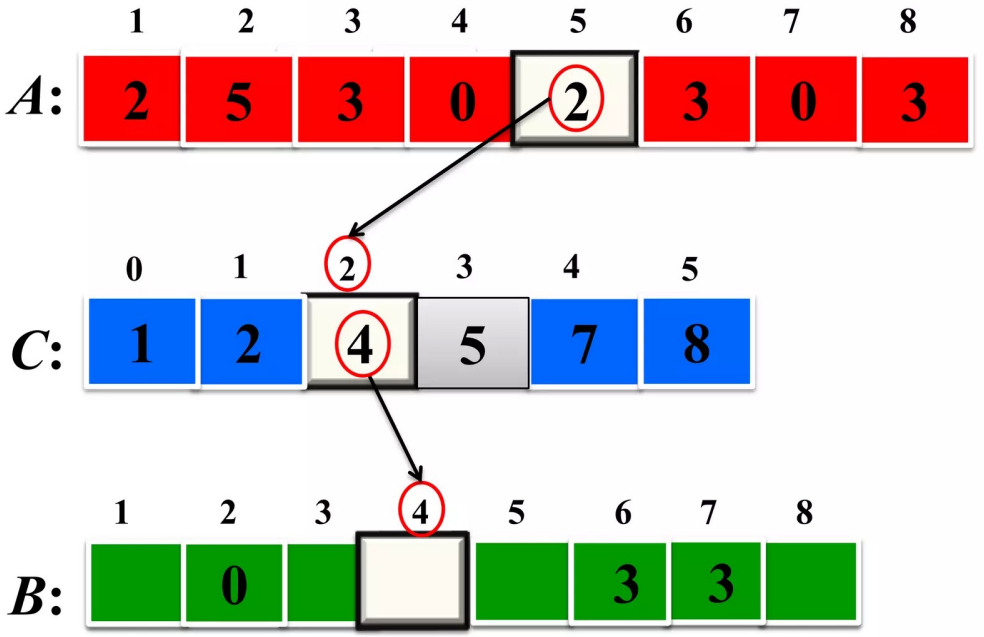
J=8, then A[j]=A[8]=3
Then C[A[j]]
= C[3]
=7
So C[A[j]] = C[A[j]] -1
=7-1=6

Counting Sort (Example)

Executing Loop 4

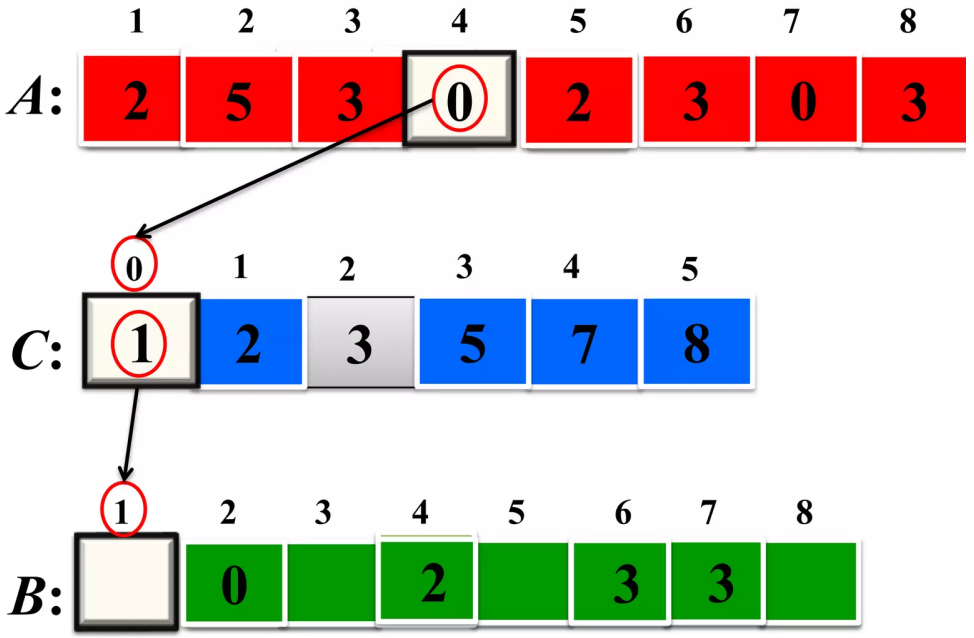


Executing Loop 4

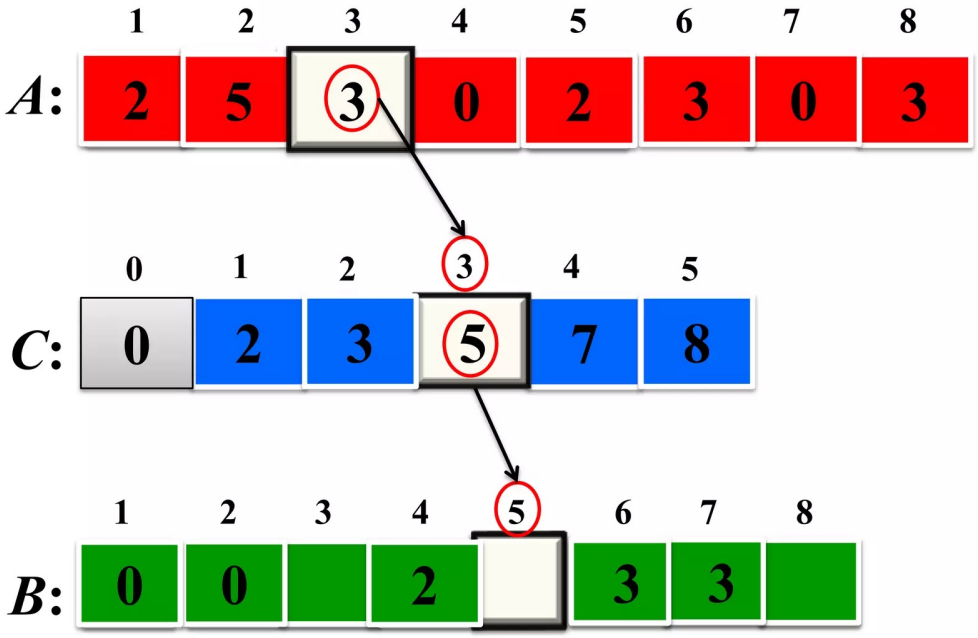


Counting Sort (Example)

Executing Loop 4

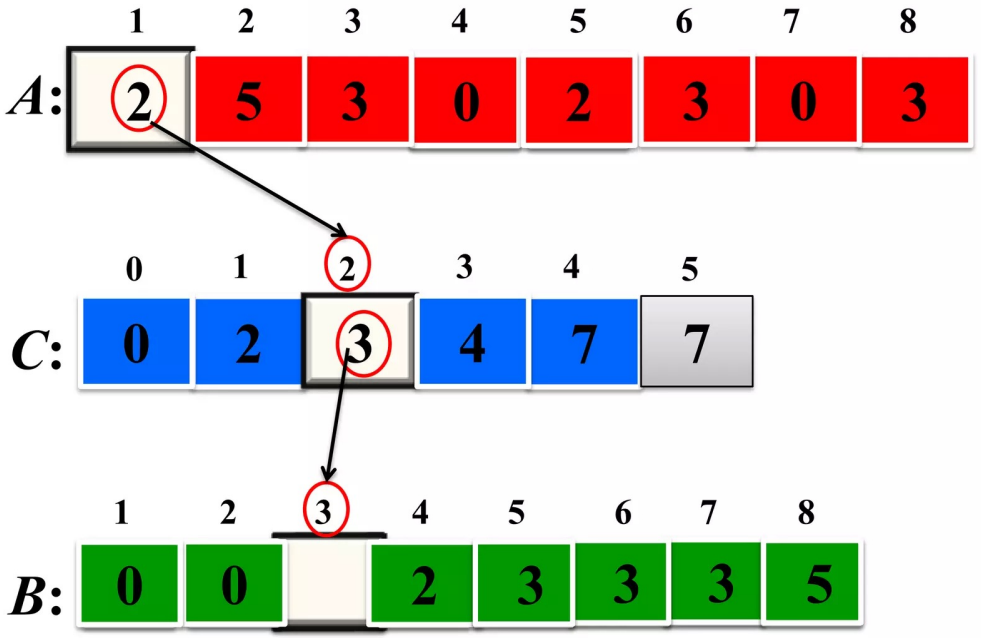


Executing Loop 4

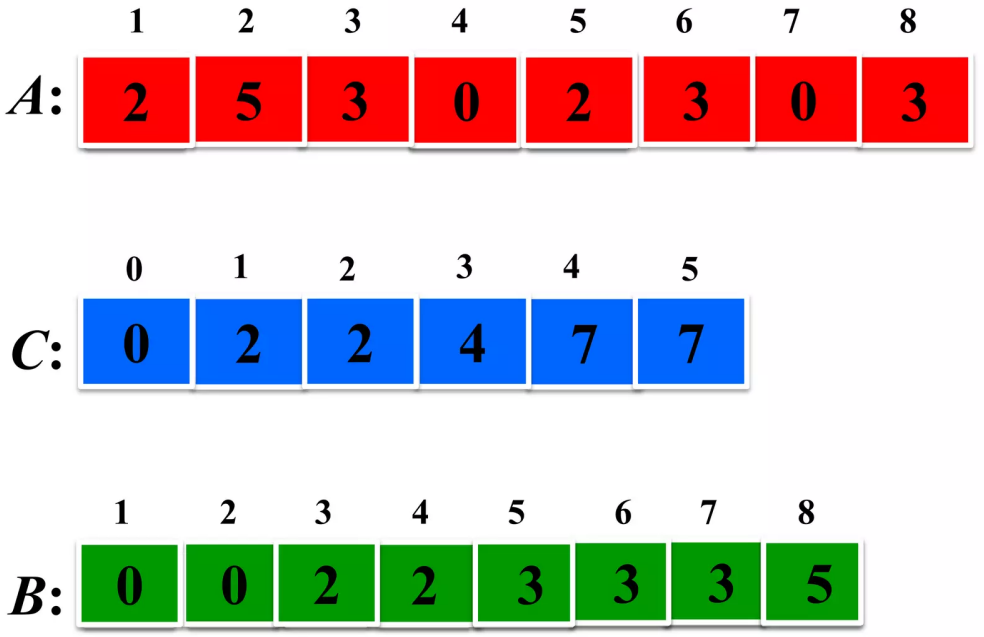


Counting Sort (Example)

Executing Loop 4



End of Loop 4

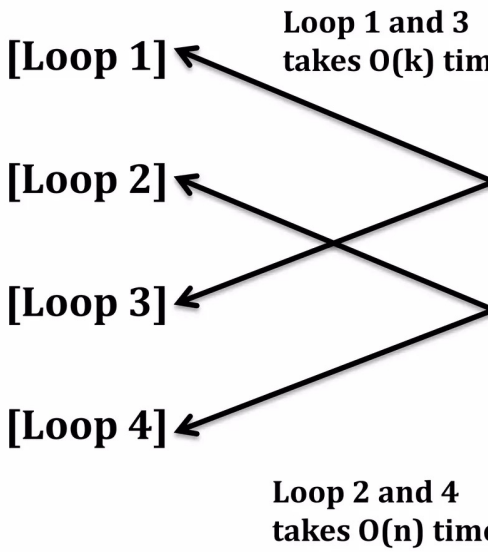


Sorted data in Array B

Counting Sort (Complexity)

Time Complexity Analysis

```
1.  Counting-Sort(A, B, k)
2.  Let C[0.....k] be a new array
3.  for i=0 to k                          [Loop 1]
4.        C[i]= 0;
5.  for j=1 to A.length  or n             [Loop 2]
6.        C[ A[j] ] = C[ A[j] ]  + 1;
7.  for i=1 to k                          [Loop 3]
8.        C[i] = C[i] + C[i-1];
9.  for j=n or A.length down to 1         [Loop 4]
10.       B[  C[ A[j] ]  ] = A[j];
11.       C[ A[j] ] = C[ A[j] ]  - 1;
```



Time Complexity Analysis

- So the counting sort takes a total time of: $O(n + k)$
- Counting sort is called stable sort.
  - A sorting algorithm is ***stable*** when numbers with the same values appear in the output array in the same order as they do in the input array.

Counting Sort

Counting Sort Review

- Why don't we always use counting sort?
  - Depends on range k of elements.
- Could we use counting sort to sort 32 bit integers? Why or why not?
- How big a k is **practical**?
  - 32-bit values? No.
  - 16-bit? Probably not.
  - 8-bit? Maybe, depending on n.
  - 4-bit? Probably (unless n is really small).

Counting Sort Review

- **Assumption:** input taken from **small** set of **numbers** of size *k*
- Basic idea:
  - Count number of elements less than you for each element.
  - This gives the position of that number – similar to selection sort.
- Pro's:
  - Fast
  - Asymptotically fast - O($n+k$)
  - Simple to code
- Con's:
  - Doesn't sort in place.
  - Requires O($n+k$) extra storage.

Bucket Sort

Bucket Sort

- **Bucket Sort** is a non-comparison-based sorting algorithm that works by distributing elements of the input into several "**buckets**" (or groups) based on their values.
- Each bucket is then sorted individually, either using a different sorting algorithm (like **Insertion Sort**) or recursively applying **Bucket Sort**.
- Finally, the contents of all the sorted buckets are concatenated to produce the final sorted result.

Steps of Bucket Sort:

1. **Divide the Range**: Divide the input data into a fixed number of equal-sized "buckets," based on the range of values.
2. **Distribute Elements**: Distribute the elements from the input array into these buckets according to their values.
3. **Sort Each Bucket**: Sort each bucket individually, either using another sorting algorithm or recursively applying Bucket Sort.
4. **Concatenate Buckets**: Concatenate the sorted elements from each bucket to get the final sorted array.

Bucket Sort

Time Complexity:

- **Best Case**: $O(n + k)$, where n is the number of elements and k is the number of buckets.
- **Average Case**: $O(n + k)$, assuming a uniform distribution of elements across buckets.
- **Worst Case**: $O(n^2)$ if all elements fall into a single bucket, and that bucket is sorted using a comparison-based algorithm like **Insertion Sort**.

Space Complexity: $O(n + k)$, where n is the number of elements and k is the number of buckets.

Suitable For:

- Bucket Sort is most efficient when the input is uniformly distributed over a known range.
- It works best when the elements are floating-point numbers or integers that fall within a specific range.

Bucket Sort (Example)

Q1. Sort the A = | 0.42 | 0.39 | 0.53 | 0.71 | 0.12 | 0.32 | using bucket sort algorithm.

Step 1: Since the numbers are in the range [0, 1], so we need to divide the range into smaller sub-ranges. Let's create 5 buckets, each with a range of 0.2.

Bucket	Range
Bucket 1	[0-0.2]
Bucket 2	[0.2-0.4]
Bucket 3	[0.4-0.6]
Bucket 4	[0.6-0.8]
Bucket 5	[0.8-1]

Since an array elements are floating-point numbers, so we need to convert the number into an index that corresponds to a bucket.

To map a floating-point number x (where $0 \leq x < 1$) into one of the buckets, we typically multiply x by the total number of buckets k and take the integer part of the result:

- ***bucket_index = floor(x * k) ;*** This step ensures that each element falls into the correct bucket, where k is the number of buckets.

Bucket Sort (Example)

- **Step 2: Distribute the Elements into Buckets**

Elements	Bucket index	Remarks
0.42	floor(0.42*5) = 2	It goes into **Bucket 3**
0.39	floor(0.39 * 5) = 1	It goes into **Bucket 2**
0.53	floor(0.53 * 5) = 2	It goes into **Bucket 3**
0.71	floor(0.71 * 5) = 3	It goes into **Bucket 4**
0.12	floor(0.12 * 5) = 0	It goes into **Bucket 1**
0.32	floor(0.32 * 5) = 1	It goes into **Bucket 2**

Now each bucket contains:

Bucket	Elements
Bucket 1	0.12
Bucket 2	0.39,0.32
Bucket 3	0.42,0.53
Bucket 4	0.71
Bucket 5	-

Bucket Sort (Example)

Step 3: Sort each Bucket using insertion sort or recursive bucket sort

Bucket	Elements	Sorted elements
Bucket 1	0.12	0.12
Bucket 2	0.39,0.32	0.32,0.39
Bucket 3	0.42,0.53	0.42,0.53
Bucket 4	0.71	0.71
Bucket 5	-	-

Step 4: Concatenate the Buckets

A=

0.12	0.32	0.39	0.42	0.53	0.71

Bucket Sort

Pseudocode

function ***bucket_sort_float****(arr, n, k):*

1. *Create k empty buckets*
2. *FOR each num in arr:*
   *bucket_index ← FLOOR(k * num)*
   // Normalize into range [0, k-1]
   INSERT num into buckets[bucket_index]
3. *FOR each bucket in buckets:*
   SORT(bucket) //Using Insertion Sort or another sorting algorithm
4. *Concatenate all sorted buckets into the final sorted array*

function ***bucket_sort***(arr, n, k):

1. Find min_value and max_value in arr
2. Compute bucket_size = (max_value - min_value) / k + 1
3. Create k empty buckets (lists)
4. Initialize an array bucket_sizes[k] to track bucket sizes
5. FOR each element num in arr:
   Compute bucket_index =FLOOR((num-min_value) / bucket_size)
   Insert num into buckets[bucket_index]
   Increment bucket_sizes[bucket_index]
6. FOR each bucket in buckets:
   Sort the bucket (using Insertion Sort or any sorting algorithm)
7. Concatenate all sorted buckets into the final sorted array

Bucket Sort

Times Complexity Analysis

Let n be the number of elements and k be the number of buckets.

1. **Find min/max values:** Scan through n elements: $O(n)$
2. **Create buckets:** Allocate k empty lists: $O(k)$
3. **Distribute elements:** Compute bucket index and insert n elements: $O(n)$
4. **Sort each bucket:** If using Insertion Sort, each bucket takes $O(m^2)$, where m is the number of elements per bucket: $O(n)$(Assuming balance distribution)
5. **Concatenate buckets:** Merge all sorted buckets: $O(n)$

Final Complexity

- **Best Case:** $O(n+k)$ (Uniformly distributed data, evenly distributed into buckets, each containing $O(1)$ elements)
- **Average Case:** $O(n+k)$ (Elements are reasonably distributed)
- **Worst Case:** $O(n^2)$ (If all elements land in one bucket, leading to $O(n^2)$ sorting time)

Radix Sort

Radix Sort

- Radix Sort is a **non-comparative, stable, linear-time sorting algorithm** that sorts numbers by processing individual digits.
- It works by distributing numbers into buckets based on each digit, starting from the least significant digit (LSD) to the most significant digit (MSD).
- Radix sort is generalization of bucket sort.
- It uses several passes of bucket sort

How Radix Sort Works

1. **Find the maximum number** in the list to determine the number of digits (**d**).
2. **Sort numbers digit by digit**
   - Starting from the least significant digit (rightmost digit), sort the numbers into the buckets based on the current digit.
   - After sorting by the current digit, gather the numbers back into a single list in the order of the buckets.
   - Repeat the process for each digit, moving towards the most significant digit (leftmost digit).
3. **Final sorted list**: After processing all digits, the list will be fully sorted.

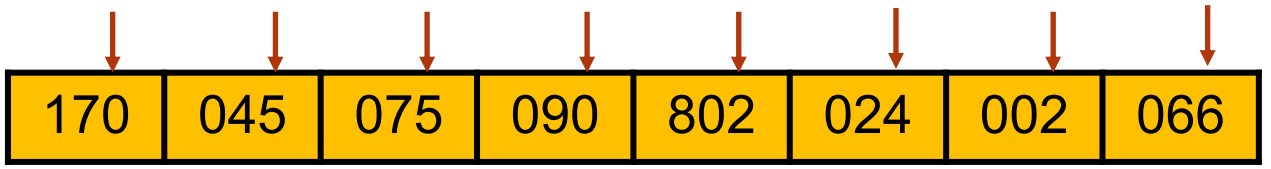
Radix Sort (Example)

Q1. Sort the A = | 170 | 45 | 75 | 90 | 802 | 24 | 2 | 66 | using Radix sort algorithm.

1. Find the maximum number: **802 (3 digits)**, so it requires 3 passes to sort.
2. Since all digits are decimal numbers, we need 10 buckets. Create and initialize it [**buckets (0-9)**].
3. Make an array elements with 3 digits numbers | 170 | 045 | 075 | 090 | 802 | 024 | 002 | 066 |

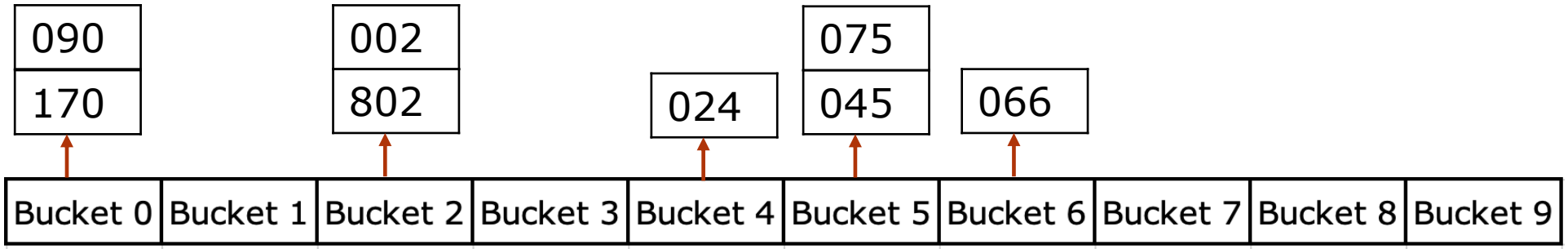
Bucket 0	Bucket 1	Bucket 2	Bucket 3	Bucket 4	Bucket 5	Bucket 6	Bucket 7	Bucket 8	Bucket 9

Radix Sort (Example)



Pass 1:

- Scan the array from left to right and find bucket number as according to LSD(one's place) and place the number in respective bucket.

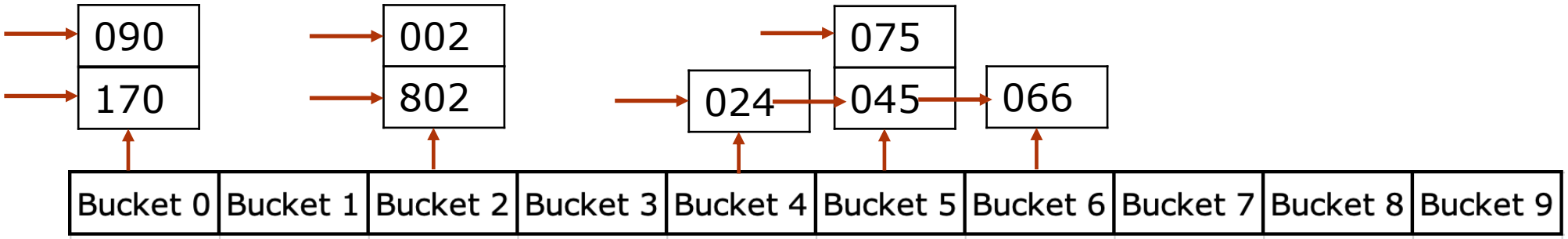


- Scan the buckets from left to right and bottom to up to construct an array after pass 1.

Radix Sort (Example)

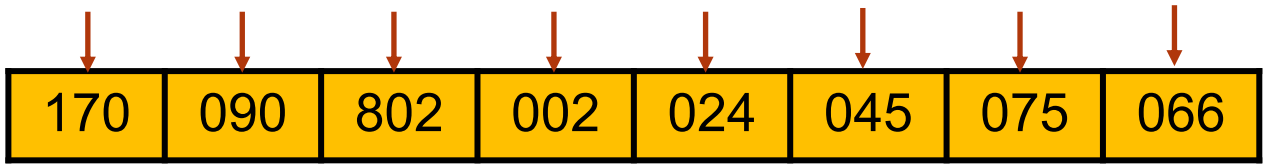
Array after Pass 1:

170	090	802	002	024	045	075	066



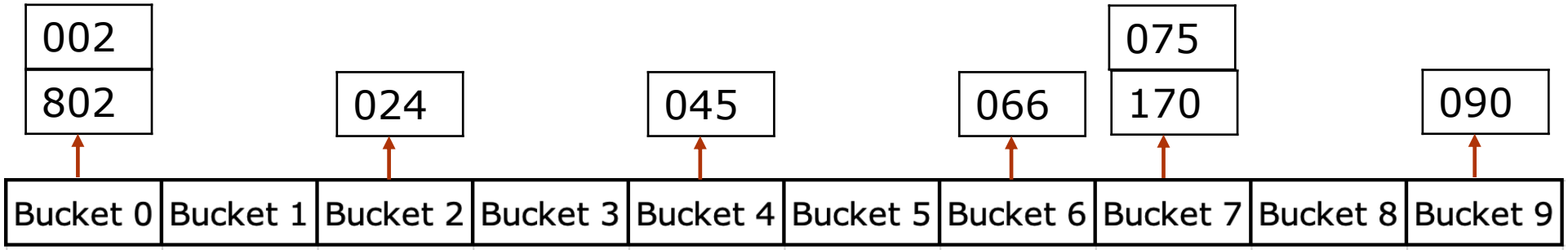
- Scan the buckets from left to right and bottom to up, construct an array after pass 1.

Radix Sort (Example)



Pass 2:

- Scan the array from left to right and find bucket number as according to Tens place and place the number in respective bucket.

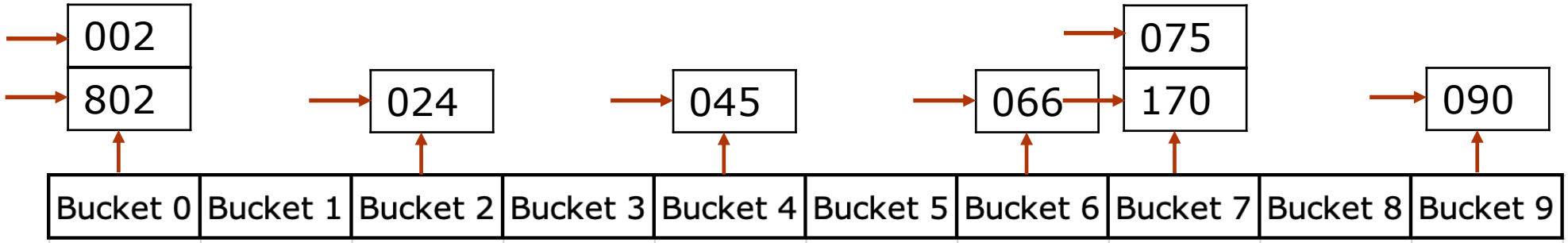


- Scan the buckets from left to right and bottom to up to construct an array after pass 2.

Radix Sort (Example)

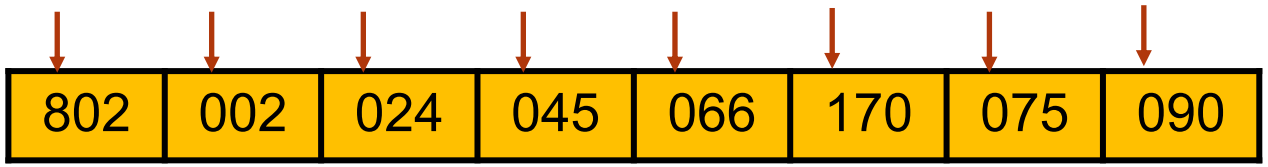
Array after Pass 2:

802	002	024	045	066	170	075	090



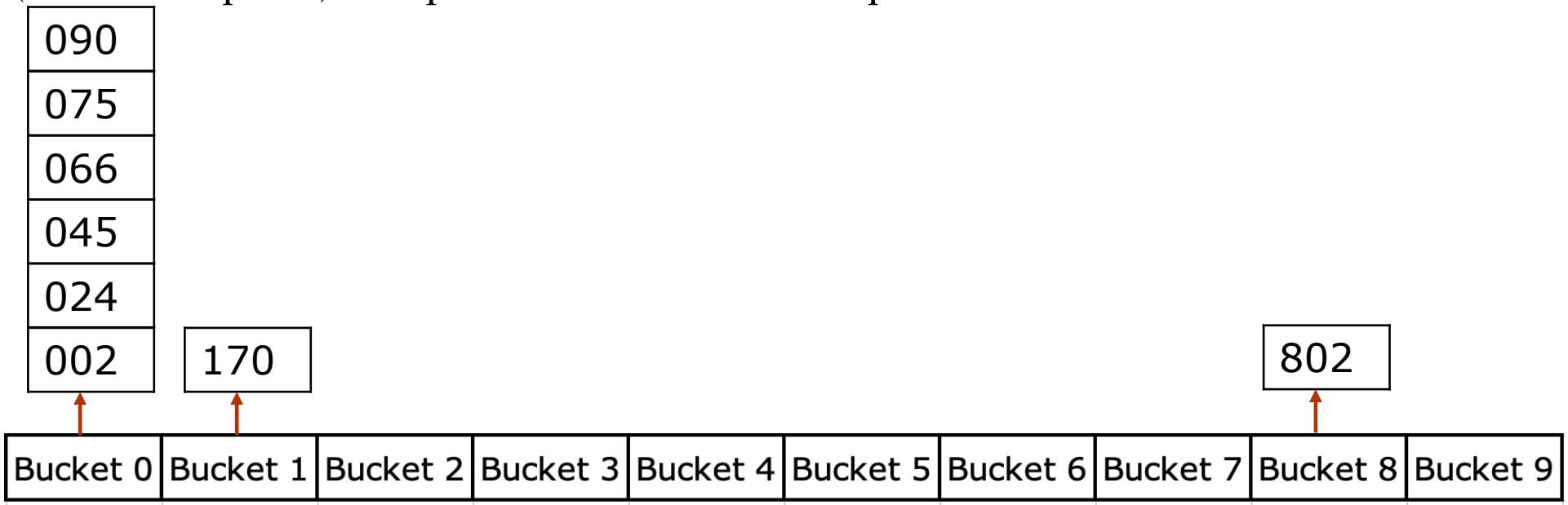
- Scan the buckets from left to right and bottom to up, construct an array after pass 2.

Radix Sort (Example)



Pass 3:

- Scan the array from left to right and find bucket number as according to MSD (Hundreds place) and place the number in respective bucket.

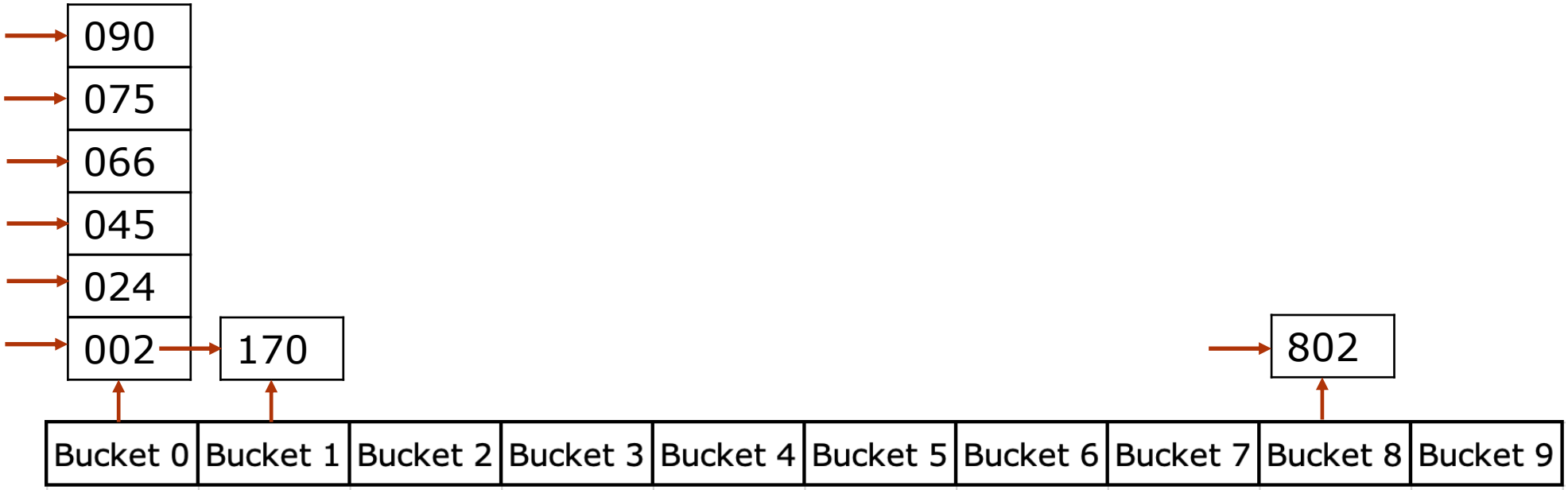


- Scan the buckets from left to right and bottom to up to construct an array after pass 3.

Radix Sort (Example)

Array after Pass 3:

002	024	045	066	075	090	170	802



The Final Sorted Array is:

2	24	45	66	75	90	170	802

Radix Sort (Time Complexity)

Assumptions: We'll assume we're sorting ***n*** items, and each item has ***k*** digits (or characters, as in our word example). We'll also assume we're using a stable sorting algorithm (like counting sort) as the sub-sort in each pass. The number of possible digits/characters is represented by ***r*** (the radix).

One Pass: A single pass of Radix Sort (sorting by one digit/character) takes ***O(n + r)*** time. This is because:

- We need to go through each of the **n** items to place them into buckets (***O(n)***).
- We need to create and manage *r* buckets (***O(r)***).

Multiple Passes: Since we have ***k*** digits/characters, we perform ***k*** passes. Therefore, the total time complexity is **O(k * (n + r)).**

Code (Radix Counting Sort)

```
void radix_sort(int array[], int n, int max_digits) {
  int i, j, k, factor;
  int* buckets = (int*) malloc(BUCKET_SIZE *
sizeof(int));
  int* temp = (int*) malloc(n * sizeof(int));
  for (k = 0, pos = 1; k < max_digits; k++, pos *=
10) {
    for (i = 0; i < BUCKET_SIZE; i++) {
      buckets[i] = 0;
    }
    for (i = 0; i < n; i++) {
      j = (array[i] / pos) % 10;
      buckets[j]++;
    }
```

```
  for (i = 1; i < BUCKET_SIZE; i++) {
      buckets[i] += buckets[i - 1];
    }
    for (i = n - 1; i >= 0; i--) {
      j = (array[i] / pos) % 10;
      temp[--buckets[j]] = array[i];
    }
    memcpy(array, temp, n * sizeof(int));
  }
  free(temp);
  free(buckets);
}
```

Try it out

Q. Illustrate the operation of RADIX-SORT on the following list of English words: COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB,BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX

Original List: COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX

- **Bucket A**: SEA, TEA
- **Bucket B**: MOB, TAB
- **Bucket G**: DOG, RUG, DIG, BIG
- **Bucket R**: BAR, EAR, TAR
- **Bucket W**: COW, ROW, NOW
- **Bucket X**: BOX, FOX

Bucket at pass 1

- **Bucket A**: TAB, BAR, EAR, TAR
- **Bucket E**: SEA, TEA
- **Bucket I**: DIG, BIG
- **Bucket O**: MOB, DOG, COW, ROW, NOW, BOX, FOX
- **Bucket U**: RUG

Bucket at pass 2

- **Bucket B**: BAR, BIG, BOX
- **Bucket C**: COW
- **Bucket D**: DIG, DOG
- **Bucket E**: EAR
- **Bucket F**: FOX
- **Bucket M**: MOB
- **Bucket N**: NOW
- **Bucket R**: ROW, RUG
- **Bucket S**: SEA
- **Bucket T**: TAB, TAR, TEA

Bucket at pass 3

After Pass 1: SEA, TEA, MOB, TAB, DOG, RUG, DIG, BIG, BAR, EAR, TAR, COW, ROW, NOW, BOX, FOX

After Pass 2: TAB, BAR, EAR, TAR, SEA, TEA, DIG, BIG, MOB, DOG, COW, ROW, NOW, BOX, FOX, RUG

After Pass 3: BAR, BIG, BOX, COW, DIG, DOG, EAR, FOX, MOB, NOW, ROW, RUG, SEA, TAB, TAR, TEA

Parallel Sorting Algorithms (Multicore and distributed system)

Parallel Sorting

- **Potential speedup**
  - Best sequential sorting algorithms (merge sort and quicksort) have (respectively worst-case and average) time complexity of $O(nlog(n))$
  - The best we can aim with a parallel sorting algorithm using n processing units is thus a time complexity of $O(nlog(n))/n = O(log(n))$ where n is the no of processing units
  - But, in general, a realistic $O(logn)$ algorithm with n processing units is a goal that is not easy to achieve with comparison-based sorting algorithms.
- **Compare-and-Exchange**
  - An operation that forms the basis of several classical sequential sorting algorithms is the compare-and-exchange (or compare-and-swap) operation.
  - In a compare-and-exchange operation, two numbers, say A and B, are compared and if they are not ordered, they are exchanged. Otherwise, they remain unchanged.

```
if (A > B) { // sorting in increasing order
   temp = A;
   A = B;
   B = temp;
}
```

Question: how can compare-and-exchange be done in parallel?

Parallel Sorting

Parallel Compare-and-Exchange : Version 1

- Suppose the compare and exchange is between the two numbers A and B, where A is held in process P_1 and B is held in process P_2.
- P_1 sends A to P_2, which then compares A and B and sends back to P_1 the min(A,B).
- It is not strictly necessary to send A back to P_1, since it already has A, but it does make it easier to have the same number of sends/receives in either case.

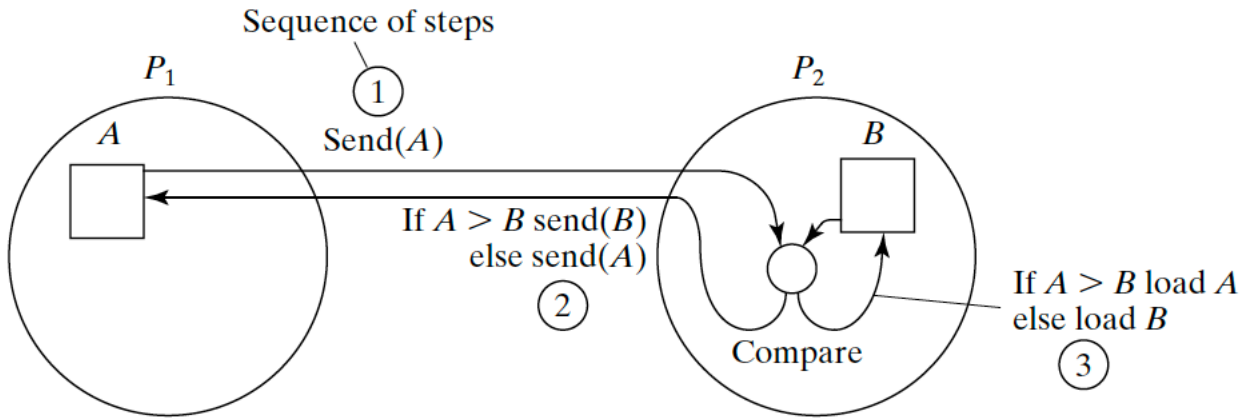


Fig: Compare and exchange on a message-passing system- Version 1.

```
Process P1
send(&A, P2);
recv(&A, P2);

Process P2
recv(&A, P1);
if (A > B) {
        send(&B, P1);
        B = A;
}
else
        send(&A, P1);
```

Parallel Sorting

Parallel Compare-and-Exchange : Version 2

- P_1 sends A to P_2 and P_2 sends B to P_1, then both perform comparisons and P_1 keeps the min(A,B) and P_2 keeps the max(A,B).

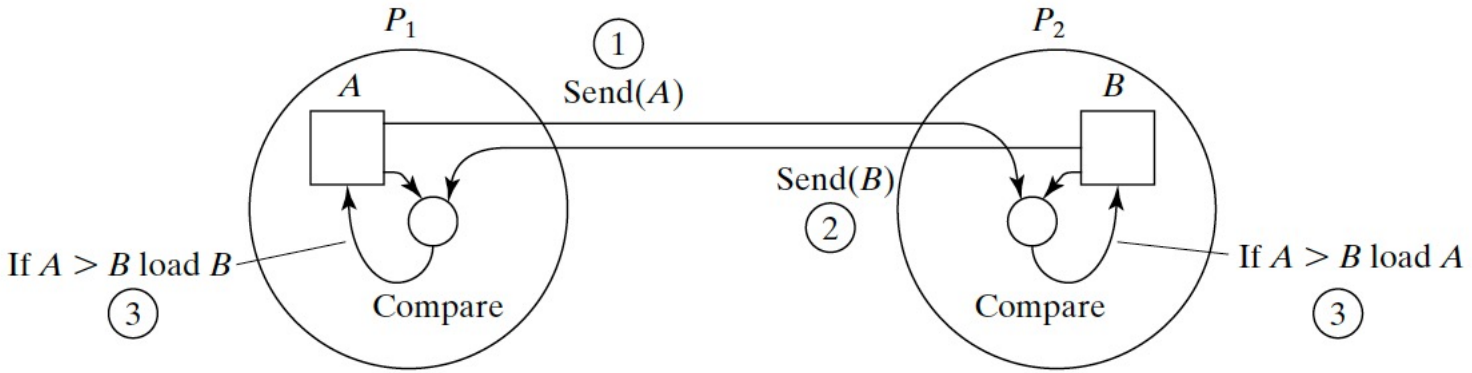


Fig: Compare and exchange on a message-passing system- Version 2.

```
Process P1
send(&A, P2);
recv(&B, P2);
if (A > B)
        A = B;
Process P2
recv(&A, P1);
send(&B, P1);
if (A > B)
        B = A;
```

- Process P_1 performs the send() first, and process P_2 performs the recv() first to avoid deadlock.

Parallel Sorting

Data Partitioning

- So far, we have assumed that there is one processing unit for each number, but normally there would be many more numbers (n) than processing units (p) and, in such cases, a list of n/p numbers would be assigned to each processing unit.
- When dealing with lists of numbers, the operation of merging two sorted lists is a common operation in sorting algorithms.

Parallel Merging-Version 1 : P_1 sends its list to P_2, which then performs the merge operation and sends back to P_1 the lower half of the merged list.

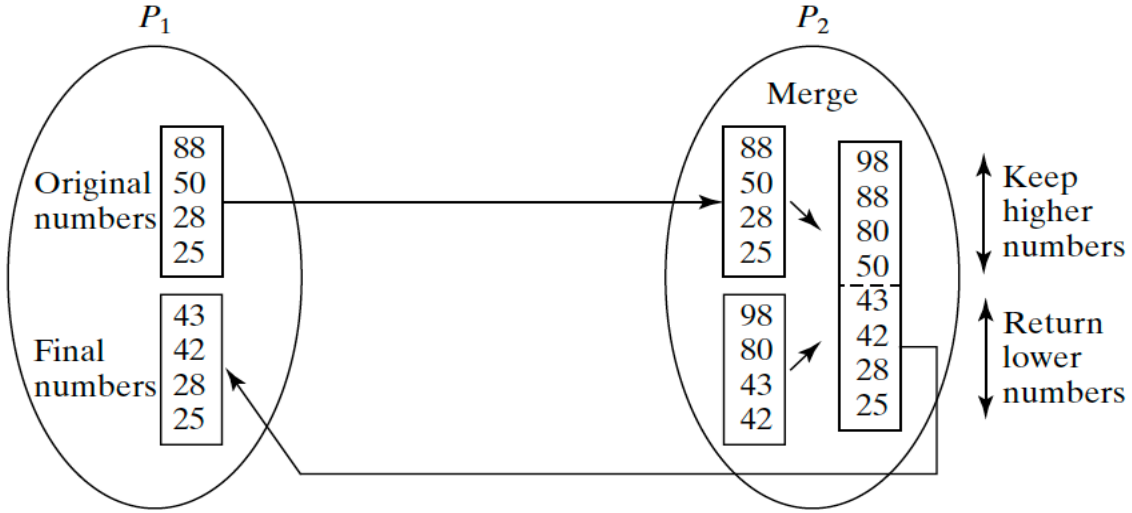


Fig: Merging two sub lists - Version 1.

Parallel Sorting

Parallel Merging-Version 2 :

- Both processing units exchange their lists, then both perform the merge operation and P_1 keeps the lower half of the merged list and P_2 keeps the higher half of the merged list.

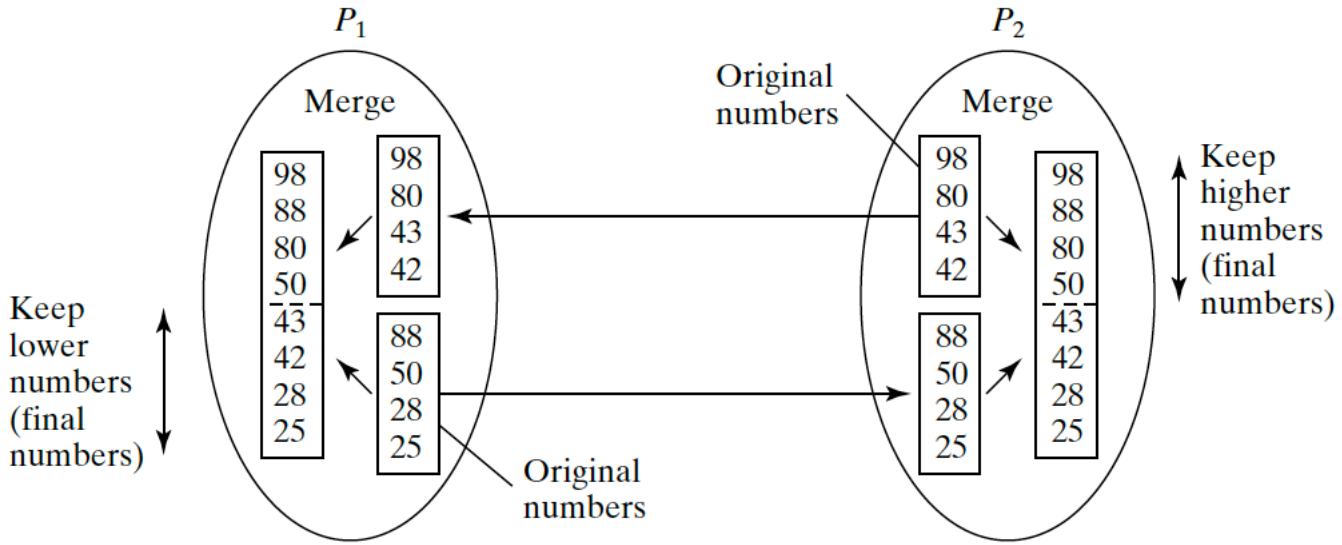


Fig: Merging two sub lists - Version 2.

Parallel Sorting

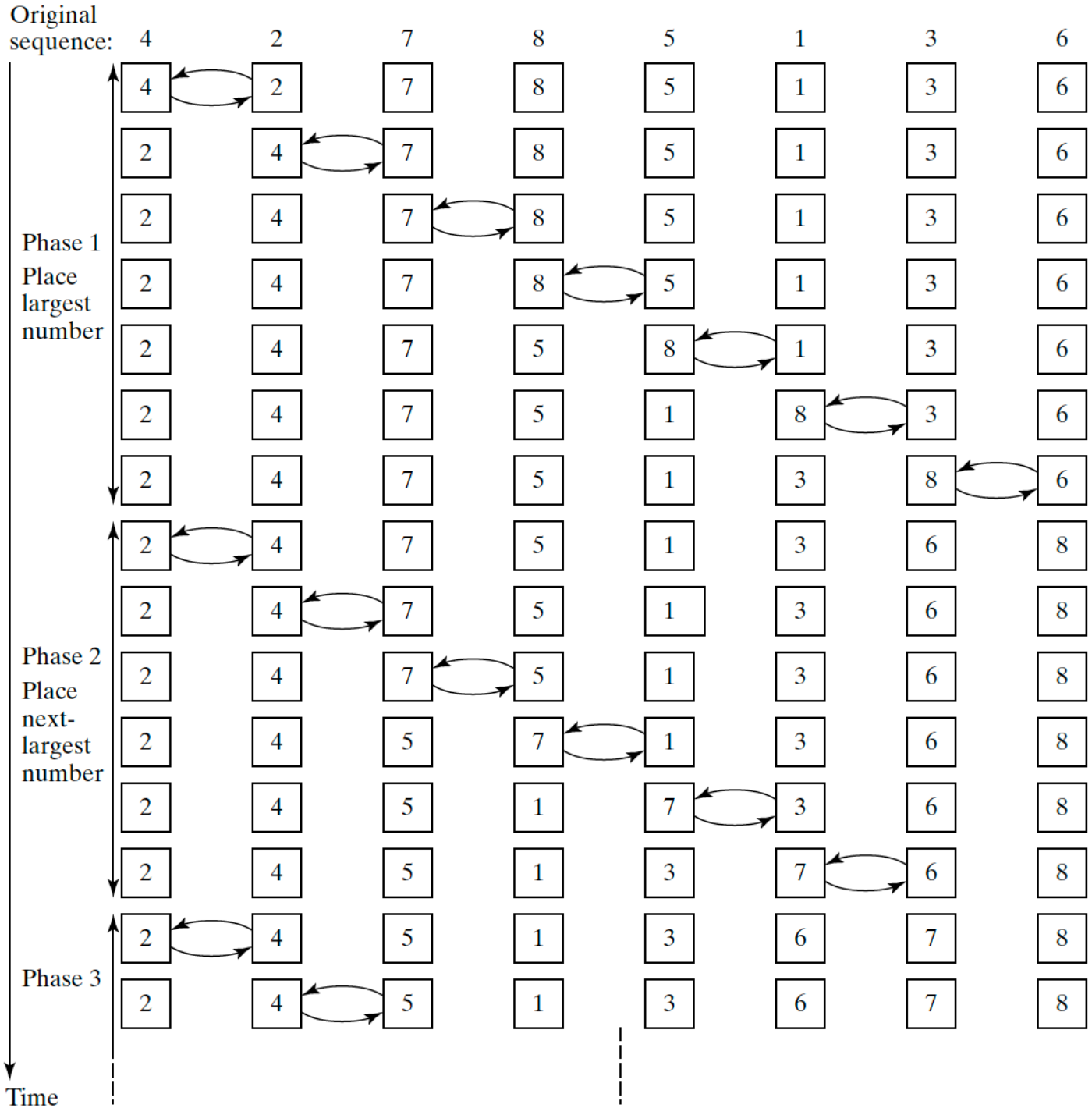
Bubble Sort

- In bubble sort, the largest number is first moved to the very end of the list by a series of compare-and-exchange operations, starting at the opposite end. The procedure repeats, stopping just before the previously positioned largest number, to get the next-largest number. In this way, the larger numbers move (like a bubble) toward the end of the list.
- The total number of compare-and-exchange operations is $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$, which corresponds to a time complexity of $\mathbf{O(n^2)}$.

Sequential Code of Bubble Sort

```
for (i = N - 1; i > 0; i--)
    for (j = 0; j < i; j++) {
        k = j + 1;
        if (a[j] > a[k]) {
            temp = a[j];
            a[j] = a[k];
            a[k] = temp;
    }
}
```

Bubble Sort


Original sequence: 4 2 7 8 5 1 3 6
Phase 1
Place largest number
Phase 2
Place next-largest number
Phase 3
Time

Parallel Sorting

Parallel Bubble Sort

- A possible idea is to run multiple iterations in a pipeline fashion, i.e., start the bubbling action of the next iteration before the preceding iteration has finished in such a way that it does not overtakes it.

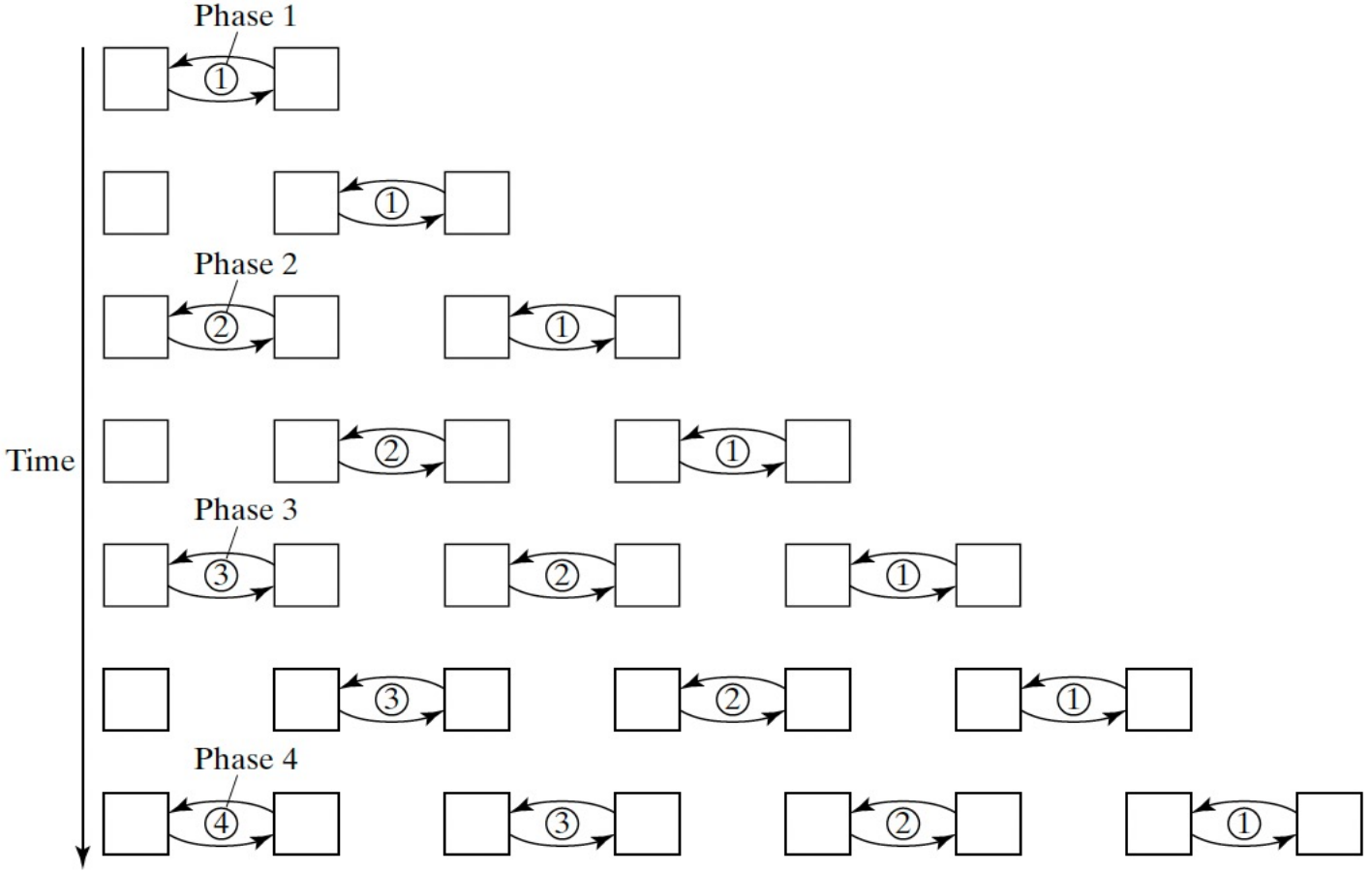


Fig: Overlapping bubble sort actions in a pipeline.

Parallel Sorting

Odd-Even Transposition Sort:

- Odd-even transposition sort is a variant of bubble sort which operates in two alternating phases:
  - Even Phase: even processes exchange values with right neighbors (P_0&P_1, P_2&P_3, …)

P_i, i = 0, 2, 4, …, n−2 (even)	*P_i, i = 1, 3, 5, …, n − 1 (odd)*
recv(&A, P_i+1);	*send(&A, P_i-1); /* even phase */*
send(&B, P_i+1);	*recv(&B, P_i-1);*
if (A < B) B = A;	*if (A < B) A = B; /* exchange */*

- Odd Phase: odd processes exchange values with right neighbors (P_1 & P_2, P_3 & P_4, …)

P_i, i = 1, 3, 5, …, n−3 (odd)	*P_i, i = 2, 4, 6, …, n − 2 (even)*
send(&A, P_i+1);	*recv(&A, P_i-1); /* even phase */*
recv(&B, P_i+1);	*send(&B, P_i-1);*
if (A > B) A = B;	*if (A > B) B = A; /* exchange */*

- For sequential programming, odd-even transposition sort has no particular advantage over normal bubble sort. However, its parallel implementation corresponds to a time complexity of **O(n)**.
- Eg: Odd-even transposition sort applied to a sequence of eight numbers, one number stored in each process, is shown in Fig (next slide).

Parallel Sorting

Odd-Even Transposition Sort:

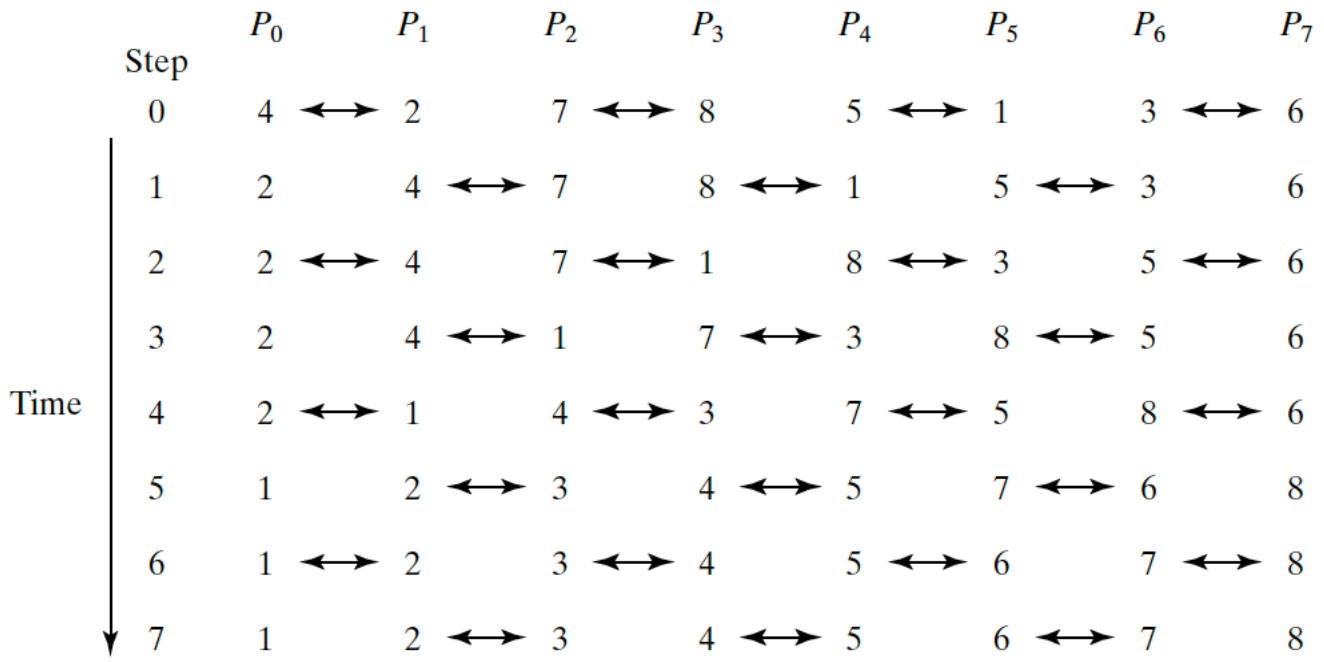


Fig: Odd-even transposition sort sorting eight numbers.

Odd-Even Transposition Sort

```
rank = process_id();
A = initial_value();
for (i = 0; i < N; i++) {
   if (i % 2 == 0) {          // even phase
      if (rank % 2 == 0) { // even process
         recv(B, rank + 1);
         send(A, rank + 1);
         A = min(A,B);
      } else {                 // odd process
            send(A, rank - 1);
            recv(B, rank - 1);
            A = max(A,B);
      }
  } else if (rank > 0 && rank < N - 1) { // odd phase
       if (rank % 2 == 0) {                  // even process
            recv(B, rank - 1);
            send(A, rank - 1);
            A = max(A,B);
       } else {                                 // odd process
             send(A, rank + 1);
             recv(B, rank + 1);
             A = min(A,B);
      }
}
```

Parallel Sorting

Sequential Merge Sort

- Merge sort is a classical sorting algorithm using a divide-and-conquer approach. The initial unsorted list is first divided in half, each half sublist is then applied the same division method until individual elements are obtained. Pairs of adjacent elements/sublists are then merged into sorted sublists until the one fully merged and sorted list is obtained.
- It is a stable sorting algorithm.
- The best, average and waste case time complexity of merge sort is *O(nlogn)*

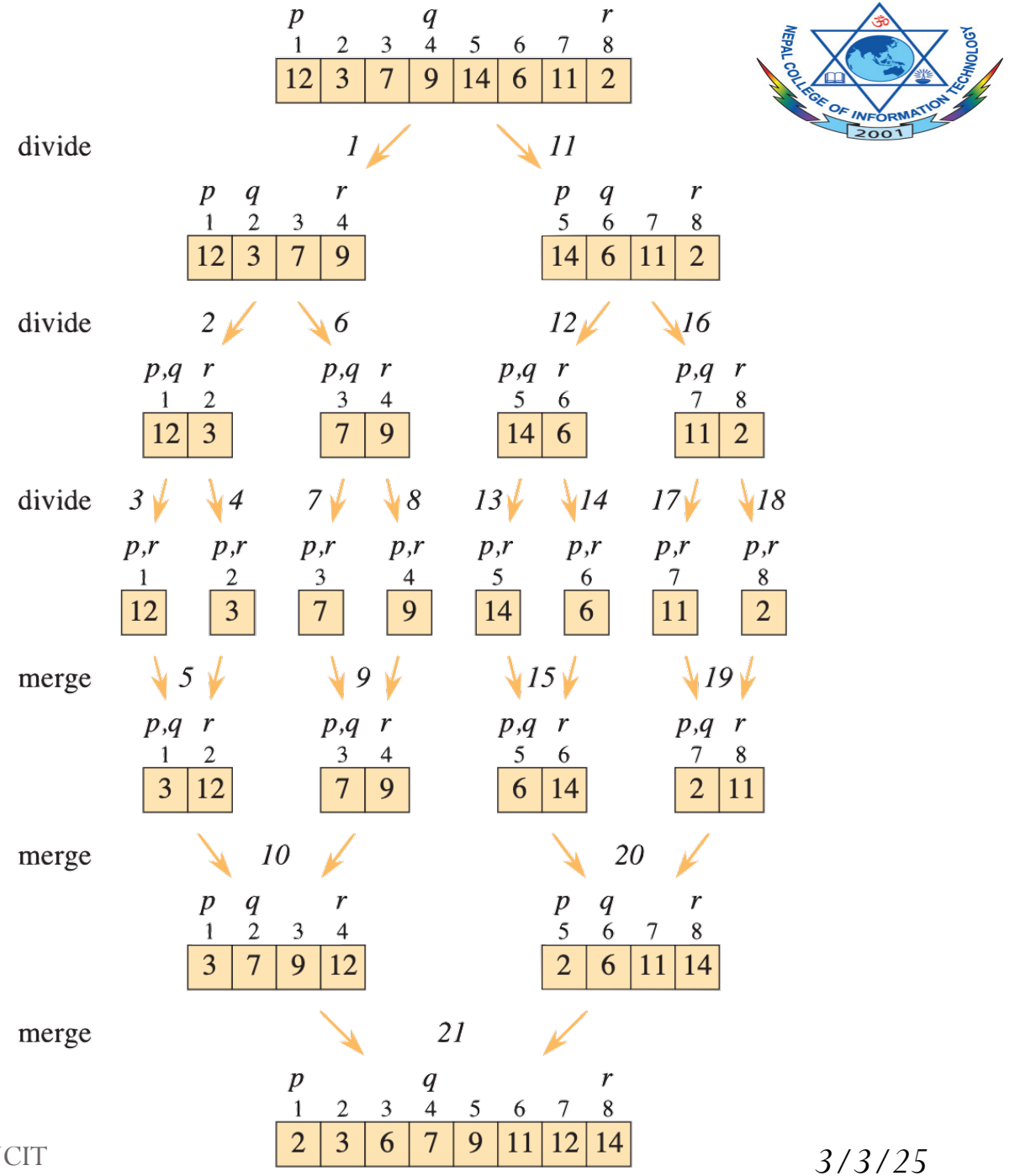
```
MERGE-SORT(A, p, r)
1  if p ≥ r                                  // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                           // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)                       // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)                   // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

Parallel Sorting

Sequential Merge Sort

- The *merge sort* algorithm closely follows the divide-and-conquer method.
- In each step, it sorts a subarray , starting with the entire array and recursing down to smaller and smaller subarrays. Here is how merge sort operates:
- **Divide** the subarray *A[p: r]* to be sorted into two adjacent subarrays, each of half the size. To do so, compute the midpoint *q* of *A[p: r]* (taking the average of *p* and *r*), and divide *A[p: r]* into subarrays *A[p:q]* and *A[q+1 :r]*.
- **Conquer** by sorting each of the two subarrays *A[p:q]* and *A[q+1: r]* recursively using merge sort.
- **Combine** by merging the two sorted subarrays *A[p: q]* and *A[q+1: r]* back into *A[p: r]* , producing the sorted answer.
- The *merge sort* reaches the base case when the subarray to be sorted has just one element.
- The key operation of the merge sort algorithm occurs in the **combine** step, which merges two adjacent, sorted subarrays using *Merge(A, p, q, r)*
- Example: **Next Slide**

```
MERGE(A, p, q, r)
 1  n_L = q − p + 1          // length of A[p : q]
 2  n_R = r − q              // length of A[q + 1 : r]
 3  let L[0 : n_L − 1] and R[0 : n_R − 1] be new arrays
 4  for i = 0 to n_L − 1     // copy A[p : q] into L[0 : n_L − 1]
 5      L[i] = A[p + i]
 6  for j = 0 to n_R − 1     // copy A[q + 1 : r] into R[0 : n_R − 1]
 7      R[j] = A[q + j + 1]
 8  i = 0                    // i indexes the smallest remaining element in L
 9  j = 0                    // j indexes the smallest remaining element in R
10  k = p                    // k indexes the location in A to fill
11  // As long as each of the arrays L and R contains an unmerged element,
    //     copy the smallest unmerged element back into A[p : r].
12  while i < n_L and j < n_R
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
18      k = k + 1
19  // Having gone through one of L and R entirely, copy the
    //     remainder of the other to the end of A[p : r].
20  while i < n_L
21      A[k] = L[i]
22      i = i + 1
23      k = k + 1
24  while j < n_R
25      A[k] = R[j]
26      j = j + 1
27      k = k + 1
```



Parallel Sorting

Parallel Merge Sort

Parallel Merge Sort is an efficient, parallelized version of the traditional Merge Sort algorithm. It leverages multiple processing units (such as multi-core CPUs or distributed systems) to divide the sorting task into smaller subproblems, which are solved concurrently, and then merges the results. This approach reduces the overall sorting time, especially for large datasets.

Steps of Parallel Merge Sort

1. **Divide**: The input array is divided into smaller subarrays. In the parallel version, this division is done across multiple processors or threads.

2. **Conquer**: Each subarray is sorted independently. Since the subarrays are processed in parallel, this step is faster than in the sequential version.

3. **Merge**: The sorted subarrays are merged back together. This step can also be parallelized by merging pairs of subarrays concurrently.

Parallel Sorting

Parallel Merge Sort

How Parallelism is Achieved?

- **Task Parallelism**: Different processing units (threads or processors) work on different parts of the array simultaneously.
- **Data Parallelism**: The same operation (e.g., sorting or merging) is applied to different parts of the array in parallel.

Advantages:

- **Faster Execution**: By utilizing multiple processors or cores, the sorting process is significantly sped up.
- **Scalability**: It scales well with the number of processors, making it suitable for large datasets and modern multi-core systems.

Challenges:

- **Synchronization**: Proper synchronization is required to ensure that all threads work correctly and merge their results without conflicts.
- **Load Balancing**: The workload must be evenly distributed among processors to avoid idle time and maximize efficiency.

Parallel Sorting

Parallel Merge Sort

Communication: In the division phase, communication only takes place as follows:

Communication at each step	Processor communication
$t_{startup} + (n/2)t_{data}$	$P_0 \rightarrow P_4$
$t_{startup} + (n/4)t_{data}$	$P_0 \rightarrow P_2; P_4 \rightarrow P_6$
$t_{startup} + (n/8)t_{data}$	$P_0 \rightarrow P_1; P_2 \rightarrow P_3; P_4 \rightarrow P_5; P_6 \rightarrow P_7$
⋮	

- In the merge phase, the reverse communications take place:

⋮	
$t_{startup} + (n/8)t_{data}$	$P_0 \leftarrow P_1; P_2 \leftarrow P_3; P_4 \leftarrow P_5; P_6 \leftarrow P_7$
$t_{startup} + (n/4)t_{data}$	$P_0 \leftarrow P_2; P_4 \leftarrow P_6$
$t_{startup} + (n/2)t_{data}$	$P_0 \leftarrow P_4$

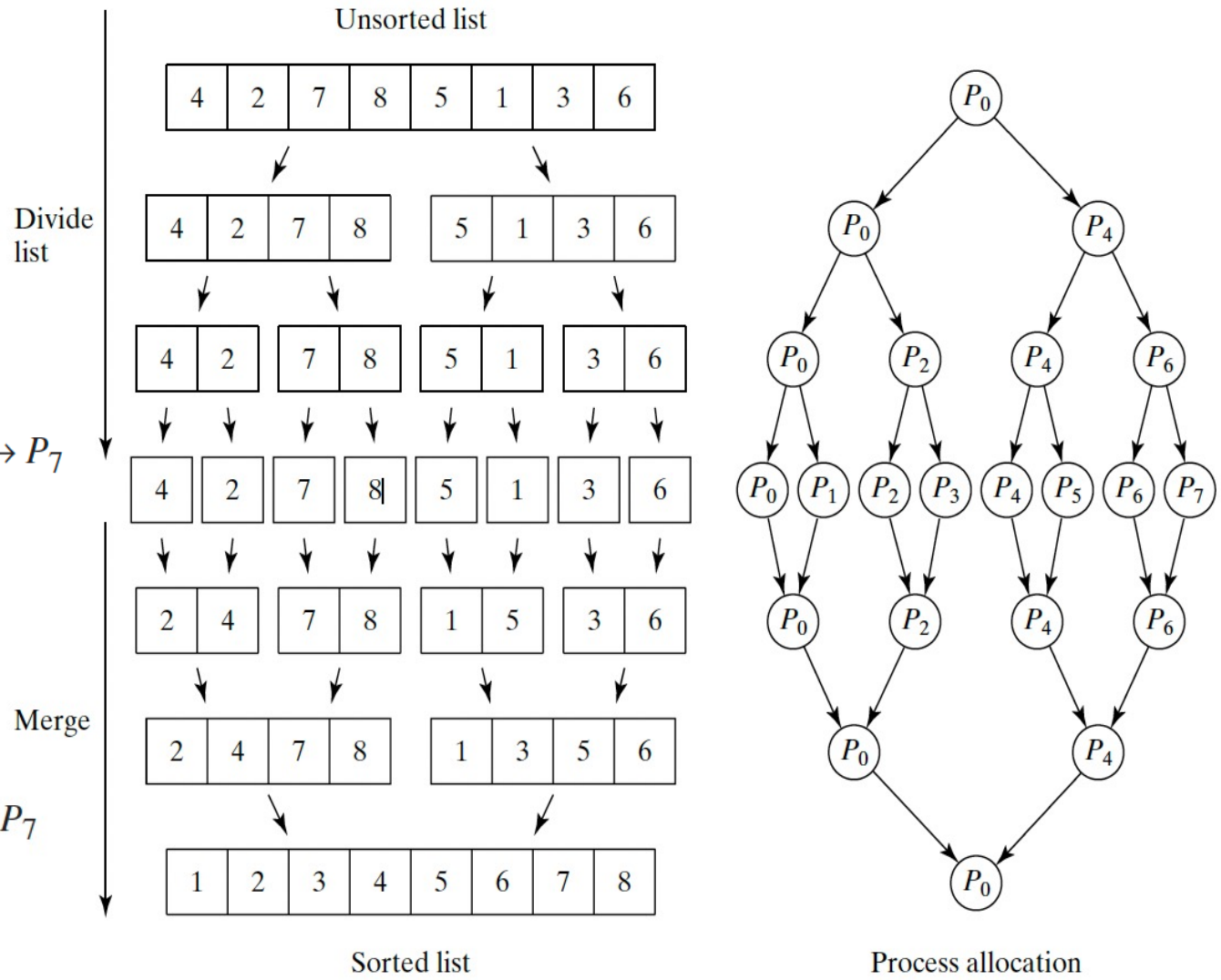


Fig: Merge sort using tree allocation of processes.

Parallel Sorting

Parallel Merge Sort

- The procedure **P-MERGE-SORT** modifies merge sort to spawn the first recursive call.
- The **P-MERGE-SORT** procedure sorts the subarray *A[p:r]*.
- After the sync statement in line 8 ensures that the two recursive spawns in lines 5 and 7 have finished, **P- MERGE-SORT** calls the **P-MERGE** procedure, a parallel merging algorithm.

```
P-MERGE-SORT(A, p, r)
 1  if p ≥ r                          // zero or one element?
 2      return
 3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
 4  // Recursively sort A[p : q] in parallel.
 5  spawn P-MERGE-SORT(A, p, q)
 6  // Recursively sort A[q + 1 : r] in parallel.
 7  spawn P-MERGE-SORT(A, q + 1, r)
 8  sync                              // wait for spawns
 9  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
10  P-MERGE(A, p, q, r)
```

Parallel Merge Sort

P-MERGE Procedure

Parameters of P-MERGE-AUX()

An input array **A** and the four *indices* p_1, r_1, p_2, r_2 specify the subarrays $A[p_1:r_1]$ and $A[p_2:r_2]$ to be merged. The array B and the index p_3 indicate that the merged result should be stored into $B[p_3:r_3]$, where $r_3 = p_3 + (r_1 - p_1) + (r_2 - p_2) + 1$. The end index r_3 of the output subarray is not needed by the pseudocode, but it helps conceptually to name the end index.

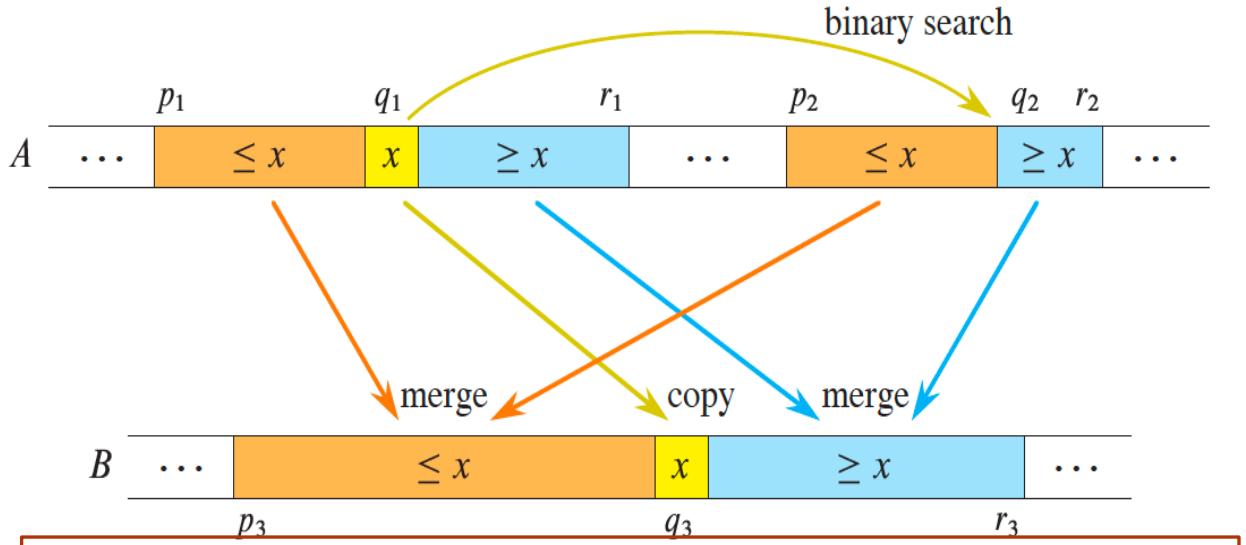
```
P-MERGE(A, p, q, r)
1  let B[p : r] be a new array                          // allocate scratch array
2  P-MERGE-AUX(A, p, q, q + 1, r, B, p)                 // merge from A into B
3  parallel for i = p to r                              // copy B back to A in parallel
4      A[i] = B[i]

P-MERGE-AUX(A, p1, r1, p2, r2, B, p3)
1  if p1 > r1 and p2 > r2                               // are both subarrays empty?
2      return
3  if r1 − p1 < r2 − p2                                 // second subarray bigger?
4      exchange p1 with p2                              // swap subarray roles
5      exchange r1 with r2
6  q1 = ⌊(p1 + r1)/2⌋                                   // midpoint of A[p1 : r1]
7  x = A[q1]                                            // median of A[p1 : r1] is pivot x
8  q2 = FIND-SPLIT-POINT(A, p2, r2, x)                  // split A[p2 : r2] around x
9  q3 = p3 + (q1 − p1) + (q2 − p2)                      // where x belongs in B ...
10 B[q3] = x                                            // ... put it there
11 // Recursively merge A[p1 : q1 − 1] and A[p2 : q2 − 1] into B[p3 : q3 − 1].
12 spawn P-MERGE-AUX(A, p1, q1 − 1, p2, q2 − 1, B, p3)
13 // Recursively merge A[q1 + 1 : r1] and A[q2 : r2] into B[q3 + 1 : r3].
14 spawn P-MERGE-AUX(A, q1 + 1, r1, q2, r2, B, q3 + 1)
15 sync                                                 // wait for spawns
```

Parallel Sorting

Parallel Merge Sort

- The heart of the algorithm is a recursive auxiliary procedure **P-MERGE-AUX** that merges two sorted subarrays of an array A into a subarray of another array B in parallel.
- The key idea of the recursive merging algorithm in ***P-MERGE-AUX*** is to split each of the two sorted subarrays of A around a pivot **x** , such that all the elements in the lower part of each subarray are at most **x** and all the elements in the upper part of each subarray are at least **x** . The procedure can then recurse in parallel on two subtasks: merging the two lower parts, and merging the two upper parts. The trick is to find a pivot **x** so that the recursion is not too lopsided.



The idea behind P-MERGE-AUX, which merges two sorted subarrays $A[p_1:r_1]$ and $A[p_2:r_2]$ into the subarray $B[p_3:r3]$ in parallel. Letting $x = A[q_1]$ (shown in yellow) be a median of $A[p_1:r_1]$ and q_2 be a place in $A[p_2:r_2]$ such that x would fall between $A[q_2\text{-}1]$ and $A[q_2]$, every element in the subarrays $A[p_1:q_1 -1]$ and $A[p_2:q_2 -1]$(shown in orange) is at most x, and every element in the subarrays $A[q_1 +1: r_1]$ and $A[q_2 + 1 : r_2]$ (shown in blue) is at least x.
To merge, compute the index q_3 where x belongs in $B[p_3:r3]$, copy x into $B[q_3]$, and then recursively merge $A[p_1: q_1 – 1]$ with $A[p_2: q_2 – 1]$ into $B[p_3 : q_3 – 1]$ and $A[q_1 + 1 : r_1]$ with $A[q_2 : r_2]$ into $B[q_3 + 1 : r_3]$.

Parallel Sorting

Parallel Merge Sort

- The recursive merging algorithm picks the pivot **x** as the middle element of the larger of the two input subarrays, which we can assume without loss of generality is $A[p_1{:}r_1]$, since otherwise, the two subarrays can just switch roles.
- That is, $x = A[q_1]$, where $q_1 = floor((p_1 + r_1)/2)$. Because $A[p_1{:}r_1]$ is sorted, x is a median of the subarray elements: every element in $A[p_1 : q_1 - 1]$ is no more than **x** , and every element in $A[q_1 + 1 : r_1]$ is no less than **x.**
- Then the algorithm finds the "*split point*" $\mathbf{q_2}$ in the smaller subarray $A[p_2{:}r_2]$ such that all the elements in $A[p_2 : q_2 - 1]$ (if any) are at most x and all the elements in $A[q_2 : r_2]$ (if any) are at least x.
- Intuitively, the subarray $A[p_2 : r_2]$ would still be sorted if x were inserted between $A[q_2 - 1]$ and $A[q_2]$. Since $A[p_2{:}r_2\ ]$ is sorted, a minor variant of binary search with x as the search key can find the split point q_2 in $O(lg\ n)$ time in the worst case.

Parallel Sorting

Parallel Merge Sort

- The serial procedure ***FIND-SPLIT-POINT (A, p, r, x)*** takes as input a sorted subarray ***A[p:r]*** and a key ***x***.
- The procedure returns a split point of ***A[p:r]***, an index ***q*** in the range $p \le q \le r + 1$ such that all the elements in ***A[p: q -1]*** (if any) are at most ***x*** and all the elements in ***A[q:r]*** (if any) are at least ***x*** .

```
FIND-SPLIT-POINT(A, p, r, x)
1  low = p                          // low end of search range
2  high = r + 1                     // high end of search range
3  while low < high                 // more than one element?
4      mid = ⌊(low + high)/2⌋       // midpoint of range
5      if x ≤ A[mid]                // is answer q ≤ mid?
6          high = mid               // narrow search to A[low : mid]
7      else low = mid + 1           // narrow search to A[mid + 1 : high]
8  return low
```

Time Complexity

- **Sequential Merge Sort**: ***O(nlogn)***
- **Parallel Merge Sort**: ***O(nlogn/p)***, where ***p*** is the number of processors.

Note: In practice, factors like communication overhead between processors, synchronization, and the specific implementation of the parallel merge algorithm can affect the actual runtime.

Parallel Sorting

Parallel Merge Sort (Simple example)

Step 1: Divide the Array Across Nodes

- Suppose we have an **array of size 16**: [15, 8, 23, 42, 4, 16, 9, 3, 25, 18, 2, 11, 7, 14, 21, 6]
- The array is **divided into 4 subarrays**, each assigned to a processing node:
  - $\mathbf{P_0}$: [15, 8, 23, 42]
  - $\mathbf{P_1}$: [4, 16, 9, 3]
  - $\mathbf{P_2}$: [25, 18, 2, 11]
  - $\mathbf{P_3}$: [7, 14, 21, 6]

Step 2: Each Node Sorts its Subarray (Local Sort)

- Each node performs a **sequential merge sort** on its assigned subarray:
  - $\mathbf{P_0}$ sorts → [8, 15, 23, 42]
  - $\mathbf{P_1}$ sorts → [3, 4, 9, 16]
  - $\mathbf{P_2}$ sorts → [2, 11, 18, 25]
  - $\mathbf{P_3}$ sorts → [6, 7, 14, 21]
- *(At this stage, each processor has a sorted list.)*

Parallel Sorting

Parallel Merge Sort (Simple example)

Step 3: Merge Sorted Subarrays in Parallel

- The sorted subarrays are merged in **logarithmic stages**:
- **Level 1: Pairwise Merge (4 → 2 groups)**
  - **P_0 & P_1** → Merge [8, 15, 23, 42] and [3, 4, 9, 16] → **New sorted list**: [3, 4, 8, 9, 15, 16, 23, 42]
  - **P_2 & P_3** → Merge [2, 11, 18, 25] and [6, 7, 14, 21] → **New sorted list**: [2, 6, 7, 11, 14, 18, 21, 25]
- *(Now, there are two sorted lists of size 8.)*

- **Level 2: Final Merge (2 → 1 group)**
  - The two lists [3, 4, 8, 9, 15, 16, 23, 42] and [2, 6, 7, 11, 14, 18, 21, 25] are merged into a **fully sorted array**:[2, 3, 4, 6, 7, 8, 9, 11, 14, 15, 16, 18, 21, 23, 25, 42]

Parallel Sorting

Sequential Quick Sort

- Quick sort is the fastest known general-purpose in-memory sorting algorithm in the average case.
- It does not require the extra array as needed by Merge sort, so it is space efficient as well.
- Quick sort applies the divide-and-conquer process for sorting an array *A[p:r]*
- **Divide**
  - Partition (rearrange) the array into two subarrays *A[p:q-1]* (the ***low*** side) and *A[q+1:r]* (the ***high*** side) such that each element in the ***low*** side of the partition is less than or equal to the pivot and each element in the ***high*** side is greater than the pivot.
  - Compute the index of the pivot as part of this partitioning procedure.
- **Conquer**
  - Conquer by calling quicksort recursively to sort each of the subarrays- *A[p:q-1]* and *A[q+1:r]*
- **Combine**
  - Nothing to combine. Since the two subarrays are already sorted, no work is needed to combine them. All the elements in *A[p:q-1]* are sorted and less than or equal to *A[q]*. All the elements in *A[q+1:r]* are also sorted and greater than or equal to *A[q]*.

Parallel Sorting

Sequential Quick Sort:

The QUICKSORT procedure implements quicksort. To sort an entire n-element array *A[1:n]*, the initial call is *QUICKSORT(A, 1,n)*.

```
QUICKSORT(A, p, r)
1  if p < r
2        // Partition the subarray around the pivot, which ends up in A[q].
3        q = PARTITION(A, p, r)
4        QUICKSORT(A, p, q − 1)  // recursively sort the low side
5        QUICKSORT(A, q + 1, r)  // recursively sort the high side

PARTITION(A, p, r)
1  x = A[r]                          // the pivot
2  i = p − 1                         // highest index into the low side
3  for j = p to r − 1                // process each element other than the pivot
4       if A[j] ≤ x                  // does this element belong on the low side?
5            i = i + 1                    // index of a new slot in the low side
6            exchange A[i] with A[j]      // put this element there
7  exchange A[i + 1] with A[r]       // pivot goes just to the right of the low side
8  return i + 1                      // new index of the pivot
```

Parallel Sorting

Sequential Quick Sort:

Partition: At the beginning of each iteration of the loop of *lines 3-6*, for any array index k, the following conditions hold:

1. if $p \le k \le i$, then $A[k] \le x$ (the tan region of Fig);
2. if $i+1 \le k \le j-1$, then $A[k] > x$ (the blue region);
3. if $k = r$, then $A[k] = x$ (the yellow region).

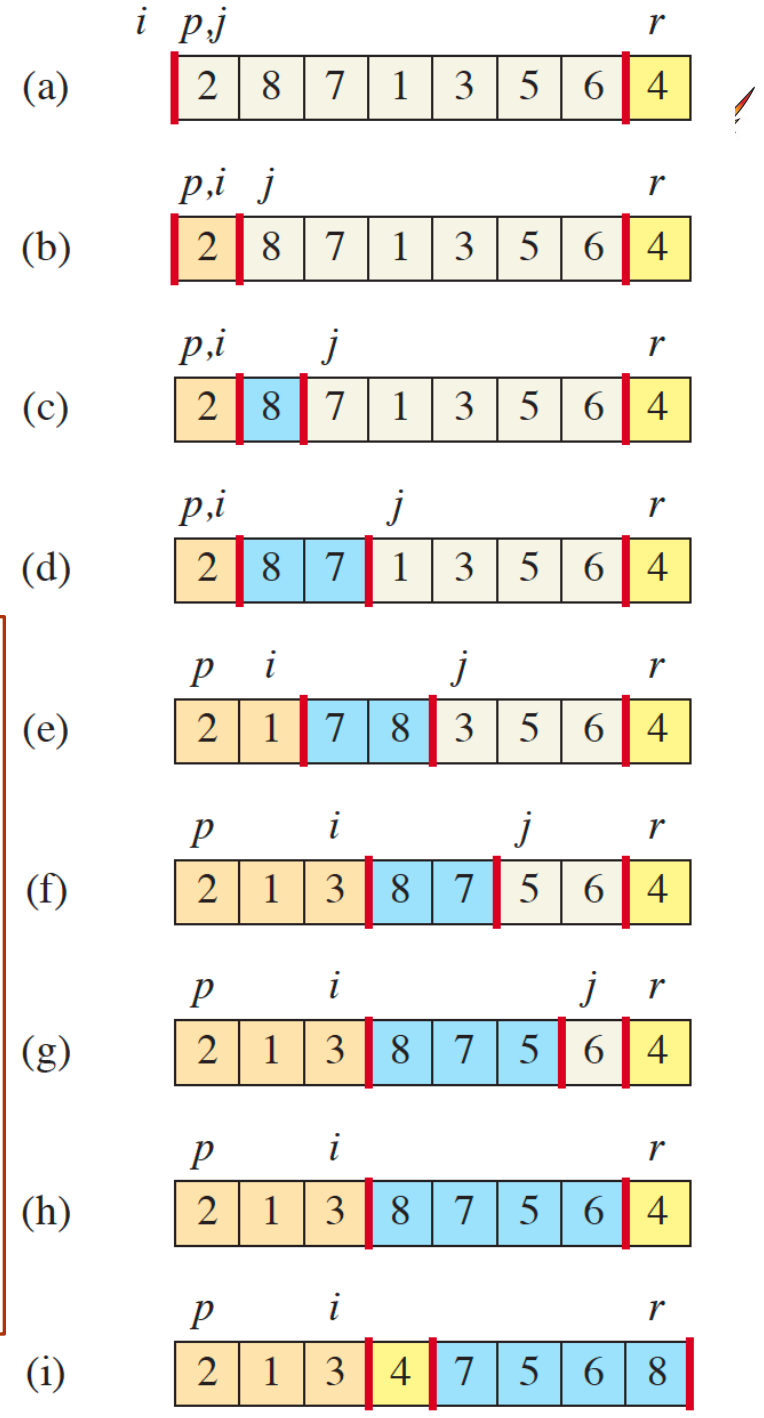


***Fig Description**: The operation of PARTITION on a sample array. Array entry A[r] becomes the pivot element x. Tan array elements all belong to the low side of the partition, with values at most x. Blue elements belong to the high side, with values greater than x. White elements have not yet been put into either side of the partition, and the yellow element is the pivot x. **(a)** The initial array and variable settings. None of the elements have been placed into either side of the partition. **(b)** The value 2 is "swapped with itself" and put into the low side. **(c)–(d)** The values 8 and 7 are placed into to high side. **(e)** The values 1 and 8 are swapped, and the low side grows. **(f)** The values 3 and 7 are swapped, and the low side grows. **(g)–(h)** The high side of the partition grows to include 5 and 6, and the loop terminates. **(i)** Line 7 swaps the pivot element so that it lies between the two sides of the partition, and line 8 returns the pivot's new index.*

Parallel Sorting

Sequential Quick Sort:

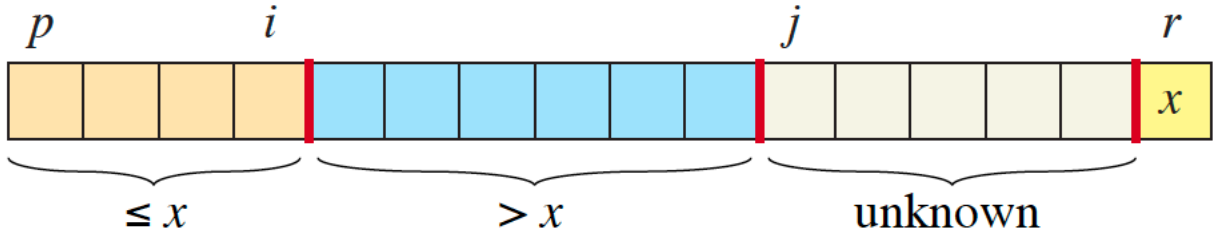


Fig: *The four regions maintained by the procedure PARTITION on a subarray **A[p: r]**. The tan values in **A[p: i]** are all less than or equal to **x**, the blue values in **A[i+1: j-1]** are all greater than **x**, the white values in **A[j:r-1]** have unknown relationships to **x**, and **A[r]= x**.*

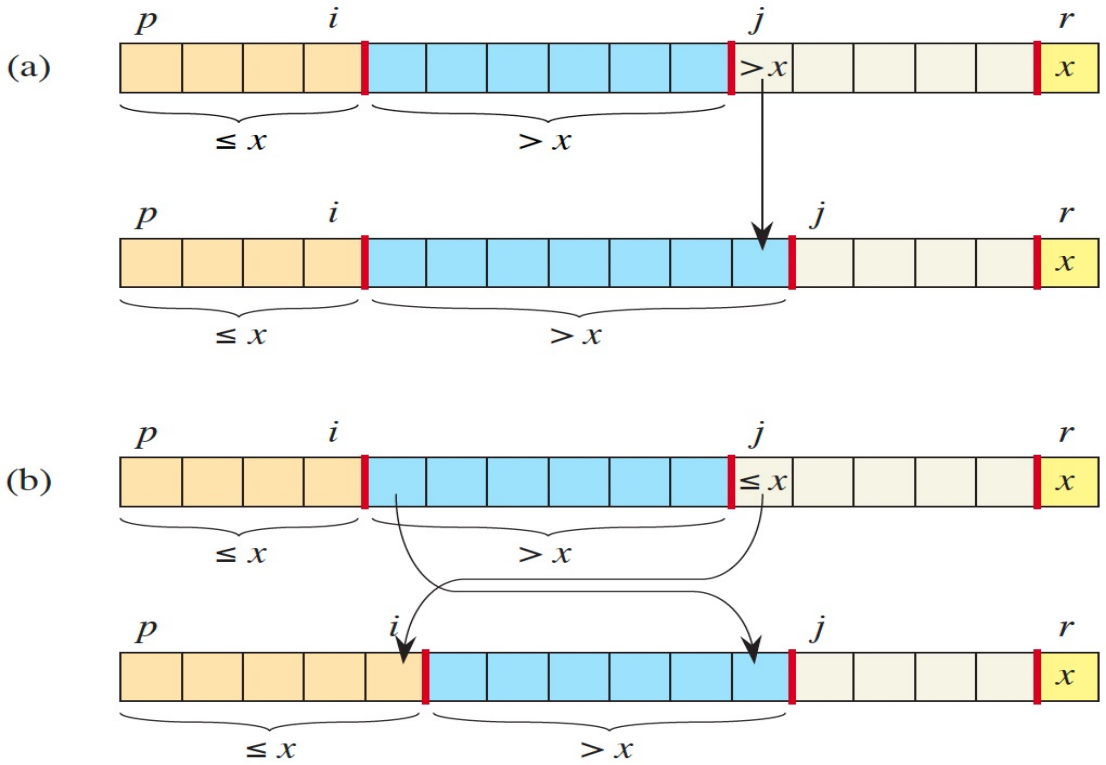


Fig: *The two cases for one iteration of procedure PARTITION. (a) If **A[j]>x** , the only action is to increment **j** , which maintains the loop invariant. (b) If **A[j] ≤ x**, index **i** is incremented, **A[i]** and **A[j]** are swapped, and then **j** is incremented. Again, the loop invariant is maintained.*

Time Complexities

- **Best/Average Case**: *O(nlogn)*[pivot element consistently divides the array into two nearly equal halves
- **Worst Case Time Complexity**: $O(n^2)$[pivot element is consistently the smallest or largest element in the array]
- **Space Complexity**: *O(logn)*(best/average),
- *O(n)*(worst)

Parallel Sorting

Sequential Quick Sort (Choosing a pivot Element)

- Choosing a pivot element is a critical step in Quick Sort because it directly affects the efficiency of the algorithm.
- The goal is to select a pivot that divides the array into two nearly equal-sized partitions, minimizing the depth of recursion and achieving optimal time complexity (***O(nlogn***)).
  - First Element as a Pivot element
  - Last element as a pivot element
  - Random pivot
  - Median-of-Three pivot
  - Median of the array (Optimal Pivot)

Parallel Sorting

Sequential Quick Sort (Choosing a pivot Element)

First/Last Element as Pivot

Choose the first/last element of the array/subarray as the pivot.

Example: Array: [8 , 2 , 6 , 9 , 3]

Pivot: 8/3

Advantages:

- Simple to implement. No additional overhead.

Disadvantages:

- Performs poorly on sorted or reverse-sorted arrays (worst case:). $O(n^2)$

Parallel Sorting

Sequential Quick Sort (Choosing a pivot Element)

Random Pivot:

- Choose a random element of the array/subarray as the pivot.
- Example: Array: [8 , 2 , 6 , 9 , 3]
- Pivot: 6 (randomly chosen)

Advantages:

- Reduces the likelihood of worst-case scenarios, as the pivot is unpredictable.
- Ensures good average-case performance.

Disadvantages:

- Requires generating random numbers, which adds a slight computational overhead.

Median-of-three Pivot:

- Select the pivot as the median of the first, middle, and last elements of the array/subarray.
- Example: Array: [8 , 2 , 6 , 9 , 3]
- First: 8, Middle: 6, last: 3
- Pivot: Median of [8, 6, 3] = 6

Advantages:

- Improves partitioning for sorted or nearly sorted arrays.
- Reduces the chance of unbalanced partitions, improving performance.

Disadvantages:

- Slightly more complex to implement due to the calculation of the median.

Parallel Sorting

Sequential Quick Sort (Choosing a pivot Element)

Median of the array (Optimal Pivot):

- Choose the median of the entire array as the pivot.
- Example: Array: [8 , 2 , 6 , 9 , 3]
- Median: 6

Advantages:

- Guarantees the best possible partitions, minimizing recursion depth.
- Ensures *O(nlogn)* time complexity in all cases.

Disadvantages:

- Finding the median requires *O(n)* time, which negates the benefits of Quick Sort's simplicity. Rarely used in practice for this reason.

Parallel Sorting

Parallel Quick Sort

- As for *mergesort*, the idea is to take advantage of the tree structure of the algorithm to assign work to processes.

- One obvious way to parallelize quicksort is to start with one processor and pass on one of the recursive calls to another processor while keeping the other recursive call to perform. This will create the now familiar tree structure like in mergesort, as in Fig.
- In this example, the pivot is carried with the left list until the final sorting action. Note that the placement of the pivot between two sublists is the final placement of this number, and thus the number need not be considered further in subsequent sorting actions. We could redraw the tree to show the pivot being withheld. Then the sorted list would be obtained by searching the tree in order.

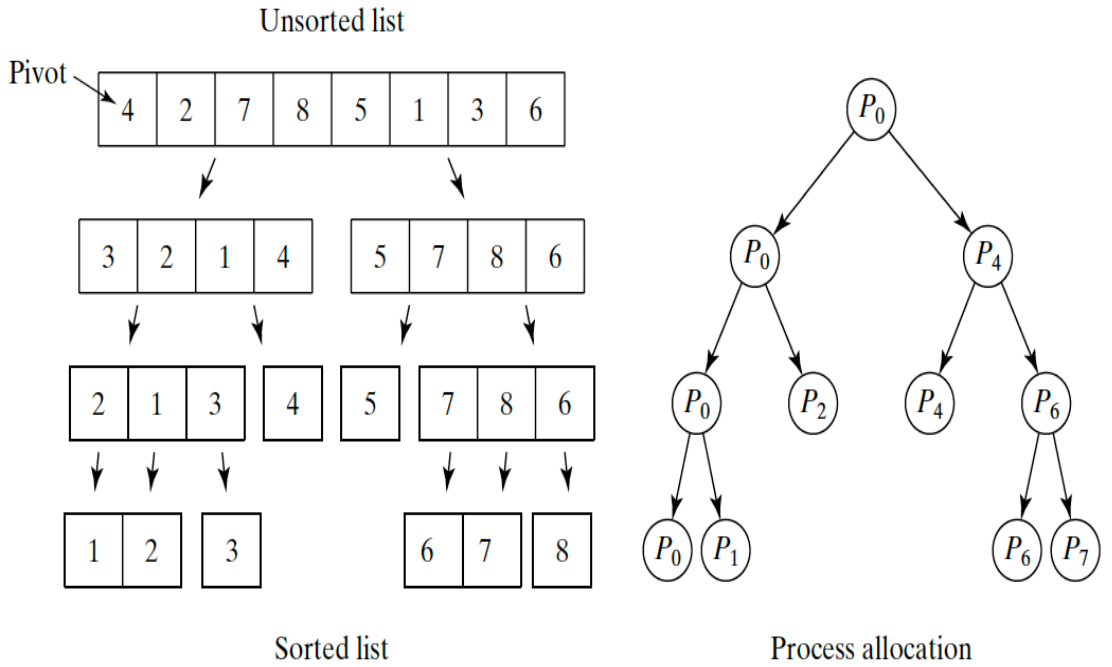


Fig: Quicksort using tree allocation of processes.

Parallel Sorting

Parallel Quick Sort

Allocating processes in a tree structure leads to two fundamental problems:

- In general, the partition tree is not perfectly balanced (selecting good pivot candidates is crucial for efficiency)
- The process of assigning work to processes seriously limits the efficient usage of the available processes (the initial partition only involves one process, then the second partition involves two processes, then four processes, and so on)

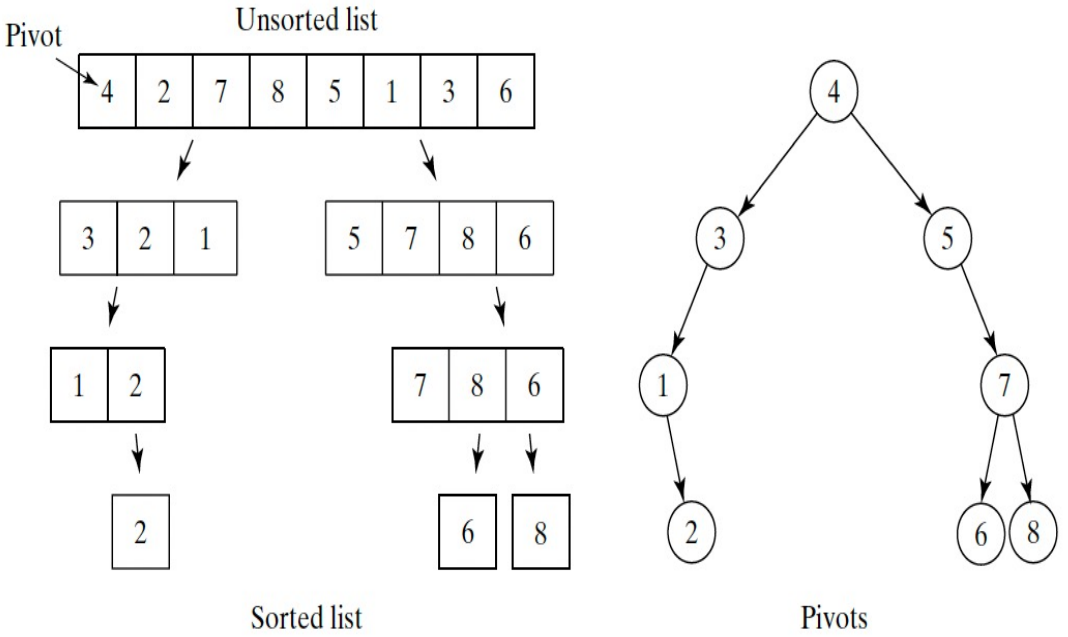


Fig: Quicksort showing pivot withheld in processes.

- Again, if we ignore communication time and consider that pivot selection is ideal, i.e., creating sublists of equal size, then it takes $\sum_{i=0}^{logn} \frac{n}{2^i} \cong 2n$ steps to obtain the final sorted list in a parallel implementation, which corresponds to a time complexity of $O(n)$. The worst-case pivot selection degenerates to a time complexity of $O(n^2)$.

Parallel Sorting

Parallel Quick Sort

Key Steps in Parallel Quick Sort

1. **Pivot Selection**: Choose a pivot (e.g., random, median-of-three) to partition the array.
2. **Partitioning:** **Rearrange** the array so that elements less than the pivot are on the left, and elements greater than the pivot are on the right.
3. **Parallel Recursive Sorting**: Spawn threads or processes to sort the left and right subarrays in parallel.
4. **Synchronization:** **Wait** for all threads or processes to complete before finishing the algorithm.

Time Complexity

1. **Best Case**: If the array is perfectly balanced at each step and there are enough processors, the time complexity can be reduced to $O(\log n)$ for the recursion depth, with each level taking $O(n)$ time for partitioning. Thus, the best-case time complexity is $O(n)$.
2. **Average Case**: With p processors, the average time complexity is $O(n \log n / p)$, assuming the workload is evenly distributed.
3. **Worst Case**: In the worst case (e.g., highly unbalanced partitions), the time complexity remains $O(n^2)$, but this can be mitigated with good pivot selection strategies.

Space Complexity

1. **Best/Average Case**: $O(\log n)$ (due to recursion stack).
2. **Worst Case**: $O(n)$ (if the recursion depth is n).

Threaded Parallel Quick Sort

```
// Function to partition the array
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}
```

```cpp
// Multi-threaded QuickSort function
void quickSort(vector<int>& arr, int low, int high, int depth = 0) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        thread leftThread, rightThread;
        bool useThreads = depth < MAX_THREADS; // Control thread depth
        if (useThreads) {
            leftThread = thread(quickSort, ref(arr), low, pivotIndex - 1, depth + 1);
            rightThread = thread(quickSort, ref(arr), pivotIndex + 1, high, depth + 1);
        } else {
            quickSort(arr, low, pivotIndex - 1, depth);
            quickSort(arr, pivotIndex + 1, high, depth);
        }
        if (useThreads) {
            leftThread.join();
            rightThread.join();
        }
    }
}
```

Parallel Sorting

Parallel Quick Sort

Work-pool Implementation

- Work-pool model can be applied to divide-and-conquer sorting algorithms such as quicksort.
- The work-pool can hold as tasks the sublists to be divided.
- First, the work-pool holds the initial unsorted list, which is given to the first processor.

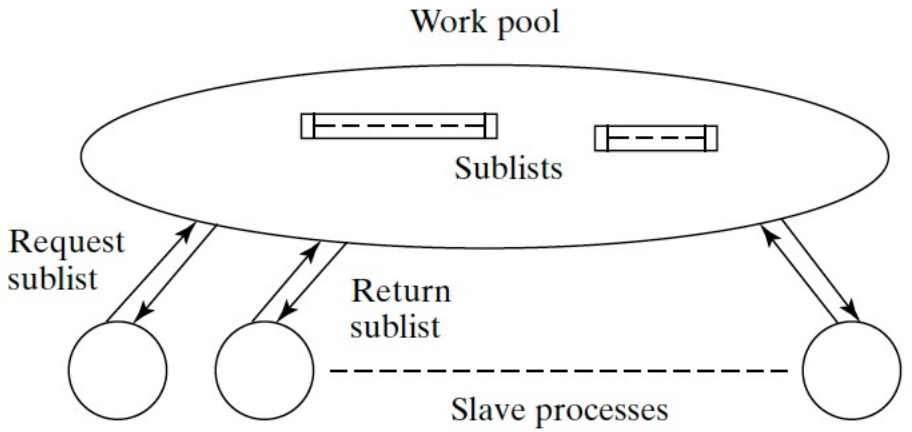


Fig: Work-pool implementation of quicksort.

- This processor divides the list into two parts.
- One part is returned to the work pool to be given to another processor, while the other part is operated upon again.
- It does not eliminate the problem of idle processors, but it deals with the case where some sublists are longer and require more work than others.

Parallel Sorting

Quick Sort partitioning algorithms (not needed)

1. Lomuto Partition Scheme (*Used here in* ***Slide 74***)

- Uses a **single** pivot (typically the last element).
- Maintains a **boundary index** to separate smaller elements.
- **Steps:**
  - Choose the pivot as the last element.
  - Initialize ***i = low-1*** (to track the correct position for the pivot).
  - Traverse from *low* to *high-1*:
    - If ***arr[j] <= pivot***, increment ***i*** and ***swap arr[i]*** and ***arr[j]***.
  - ***Swap arr[i+1]*** with the pivot (placing the pivot at its correct position).
- **Time Complexity:** **O(n)** for partitioning.
- **Space Complexity:** ***O(1)*** (in-place).

Lomuto Partition Scheme

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
                swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
 return (i + 1);
}
```

Parallel Sorting

Quick Sort partitioning algorithms (not needed)

2. Hoare Partition Scheme

- Uses **two pointers** to swap elements around a chosen pivot.
- **Steps:**
  - Select a ***pivot*** (typically the first element).
  - Use two pointers: ***i = low (left)*** and ***j = high (right).***
  - Move ***i*** right while ***arr[i] < pivot*** and move ***j*** left while ***arr[j] > pivot***.
  - ***Swap arr[i]*** and ***arr[j]*** when ***i < j***.
  - Stop when ***i >= j***, *return j* as the partition index.

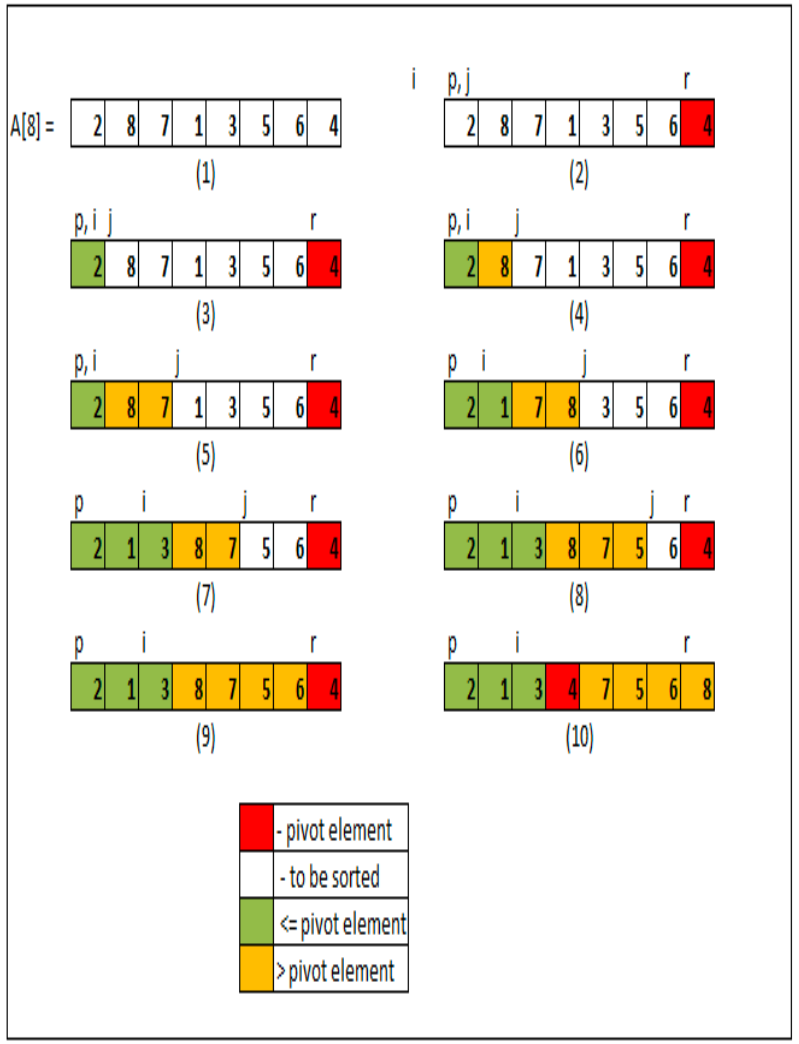
Time Complexity: O(n).

More efficient than Lomuto (fewer swaps).

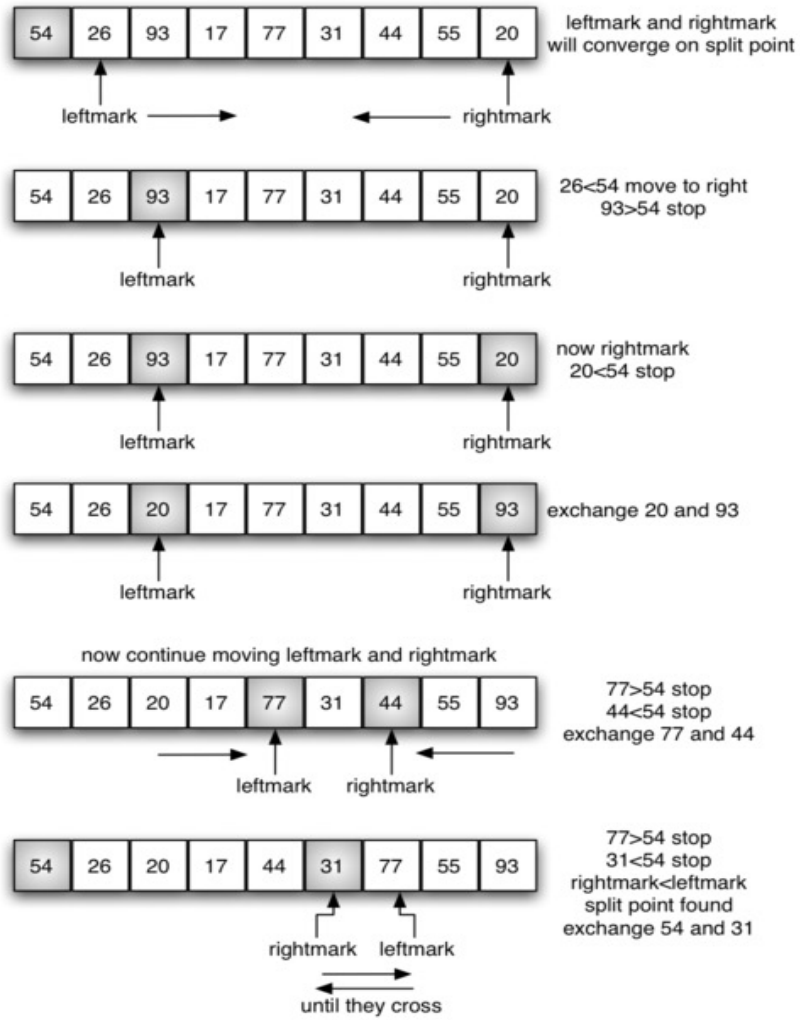
```
HOARE-PARTITION(A, p, r)
 1  x = A[p]
 2  i = p − 1
 3  j = r + 1
 4  while TRUE
 5      repeat
 6          j = j − 1
 7      until A[j] ≤ x
 8      repeat
 9          i = i + 1
10      until A[i] ≥ x
11      if i < j
12          exchange A[i] with A[j]
13      else return j
```

Quick Sort partitioning algorithms (not needed)

Lomuto Partition Scheme



Hoare Partition Scheme



Parallel Sorting

Quick Sort partitioning algorithms (not needed)

3. Median-of-Three Partitioning

- Selects the pivot as the median of three values: ***low, mid, high***.
- Reduces the chances of worst-case performance for already sorted or reverse-sorted arrays.
- Typically combined with **Hoare's partitioning**.

4. Randomized Partitioning

- Randomly selects the pivot before applying **Lomuto's** or **Hoare's** method.
- Reduces the likelihood of hitting the worst-case $O(n^2)$ scenario.

Randomize Partition Scheme

```
int randomPartition(int arr[], int low, int high) {
    int randIndex = low + rand() % (high - low);
    swap(arr[randIndex], arr[high]);
    return partition(arr, low, high);
}
```

1. **Lomuto's method** is simpler but makes more swaps.
2. **Hoare's method** is generally faster with fewer swaps.
3. **Median-of-Three** helps with partially sorted arrays.
4. **Randomized partitioning** is useful for unpredictable input.

Parallel Sorting

Odd-even Merge

- Odd-even merge is a merging algorithm for sorted lists which can merge two sorted lists into one sorted list. It works as follows:
- Let $A = [a_0, \ldots, a_{n-1}]$ and $B = [b_0, \ldots, b_{n-1}]$ be two sorted lists
- Consider $E(A) = [a_0, a_2, \ldots, a_{n-2}]$, $E(B) = [b_0, b_2, \ldots, b_{n-2}]$ and $O(A) = [a_1, a_3, \ldots, a_{n-1}]$, $O(B) = [b_1, b_3, \ldots, b_{n-1}]$ as the sublists with the elements of A and B in even and odd positions, respectively
  - Recursively merge $E(A)$ with $E(B)$ to obtain C
  - Recursively merge $O(A)$ with $O(B)$ to obtain D
  - Interleave C with D to get an almost-sorted list E
  - Rearrange unordered neighbors (using compare-and-exchange operations) to completely sort E

Parallel Sorting

Odd-even Merge

- Let A = [2, 4, 5, 8] and B = [1, 3, 6, 7] then:
  - E(A) = [2, 5] and O(A) = [4, 8]
  - E(B) = [1, 6] and O(B) = [3, 7]
- Odd-even merge algorithm:
  - Recursively merge E(A) with E(B) to obtain C
    - C = odd_even_merge([2, 5], [1, 6]) = [1, 2, 5, 6]
  - Recursively merge O(A) with O(B) to obtain D
    - D = odd_even_merge([4, 8], [3, 7]) = [3, 4, 7, 8]
  - Interleave C with D to obtain E
    - E = [1, 3, 2, 4, 5, 7, 6, 8]
  - Rearrange unordered neighbors (c, d) as ***(min(c, d), max(c, d))***
    - E = [1, 2, 3, 4, 5, 6, 7, 8]

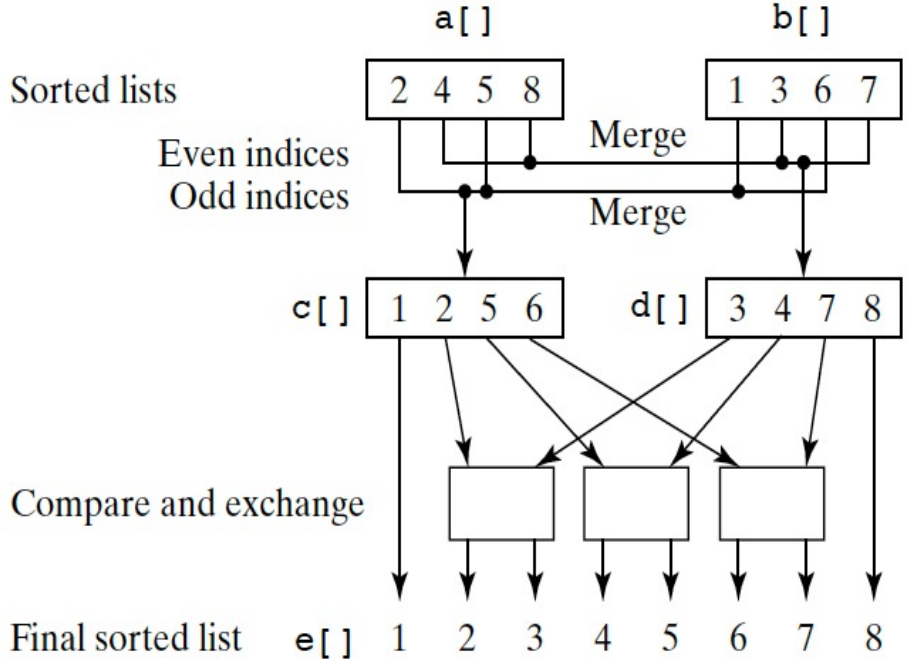


Fig: Odd-even merging of two sorted lists.

Parallel Sorting

Odd-even Merge

- In summary, merging two sorted lists of size ***n*** requires:
  - Two recursive calls to ***odd_even_merge()*** with each call merging two sorted sublists of size ***n/2***
  - ***n − 1*** compare-and-exchange operations

- For example, the call to ***odd_even_merge([2, 4, 5, 8], [1, 3, 6, 7])*** leads to:
  - One call to odd_even_merge([2, 5], [1, 6])
  - Another call to odd_even_merge([4, 8], [3, 7])
  - 3 compare-and-exchange operations

Parallel Sorting

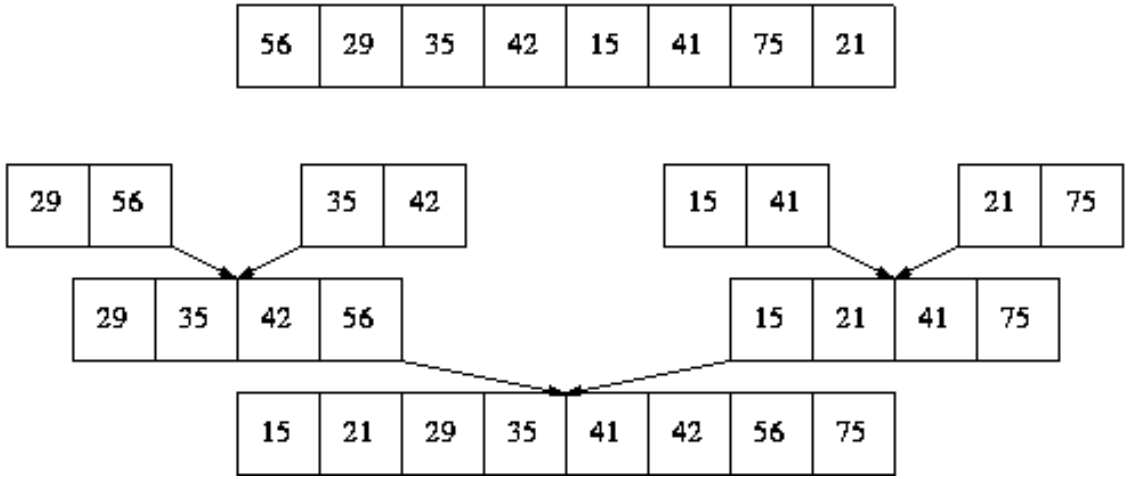
Parallel Odd-even Merge sort

- For each *odd_even_merge()* call with lists of size *n*, we can:
  - Run in parallel the two recursive calls to *odd_even_merge()* for merging sorted sublists of size *n/2*
  - Run in parallel the *n−1* compare-and-exchange operations
- Since in total there are *log(n)−1* recursive merging steps, it takes *log(n) + 1* parallel steps to obtain the final sorted list in a parallel implementation, which corresponds to a time complexity of *O(log(n)):*
  - 2 parallel steps for the compare-and-exchange operations of the last recursive calls to *odd_even_merge()* (sublists of size 2)
  - *log(n)−1* parallel steps for the remaining compare-and-exchange operations

Parallel Sorting

Parallel Odd-even Merge sort

- Odd-even mergesort is a parallel sorting algorithm based on the recursive application of the odd-even merge algorithm that merges sorted sublists bottom up – starting with sublists of size 2 and merging them into bigger sublists-until the final sorted list is obtained.



- How many steps does the algorithm?
  - Since in total there are $log(n)$ sorting steps and each step (odd-even merge algorithm) corresponds to a time complexity of $O(log(n))$, the odd-even mergesort time complexity is thus $O(log^2 n)$.

Parallel Sorting

Bitonic Merge Sort

- The basis of bitonic mergesort is the bitonic sequence, a list having specific properties that are exploited by the sorting algorithm.
- A sequence is considered bitonic if it contains two sequences, one increasing and one decreasing, such that: $a_1 < a_2 < \ldots < a_{i-1} < a_i > a_{i+1} > a_{i+2} > \ldots > a_n$
- for some value i $(0 \leq i \leq n)$. A sequence is also considered bitonic if the property is attained by shifting the numbers cyclically (left or right).

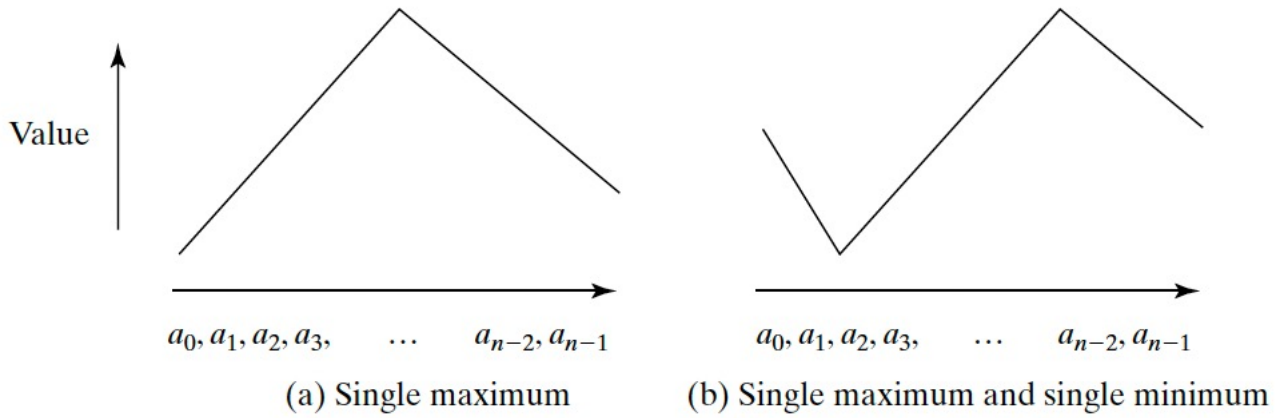


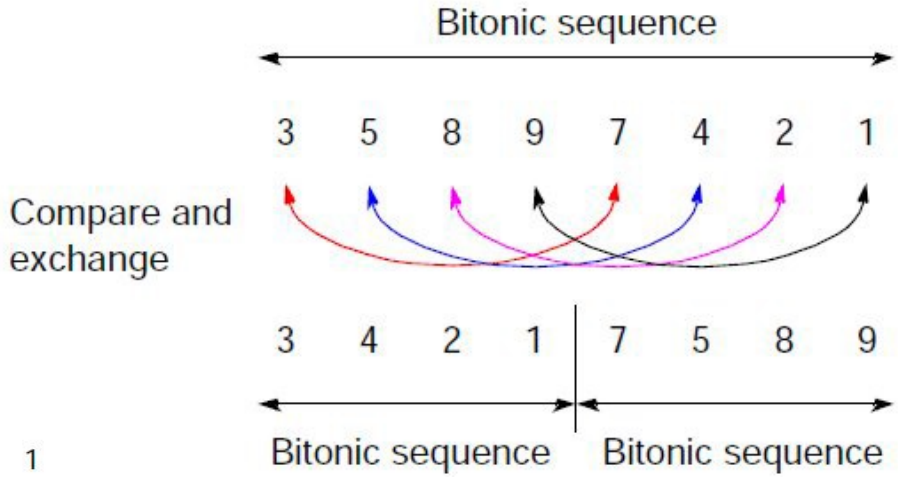
Fig: Bitonic Sequence

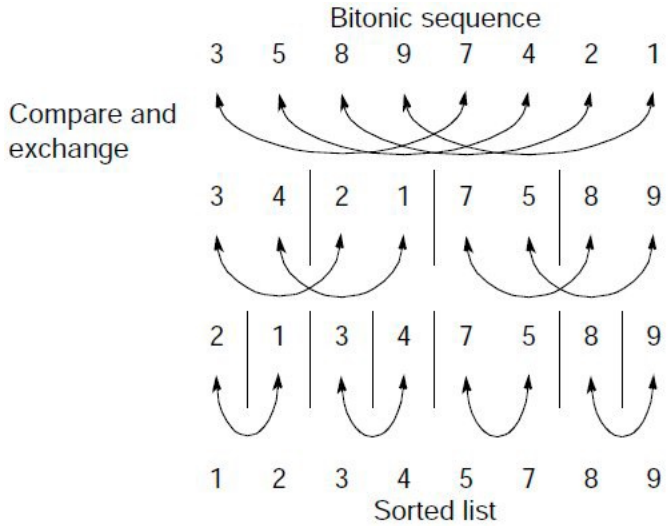
Parallel Sorting

Bitonic Merge Sort

- A special characteristic of bitonic sequences is that if we perform a compare-and-exchange operation with elements a_i and $a_{i+n/2}$ for all i $(0 \leq i \leq n/2)$ in a sequence of size n, we obtain two bitonic sequences in which all the values in one sequence are smaller than the values of the other.

- In addition to both sequences being bitonic sequences, all values in the left sequence are less than all the values in the right sequence. Hence, if we apply recursively these compare-and-exchange operations to a given bitonic sequence, we will get a sorted sequence.



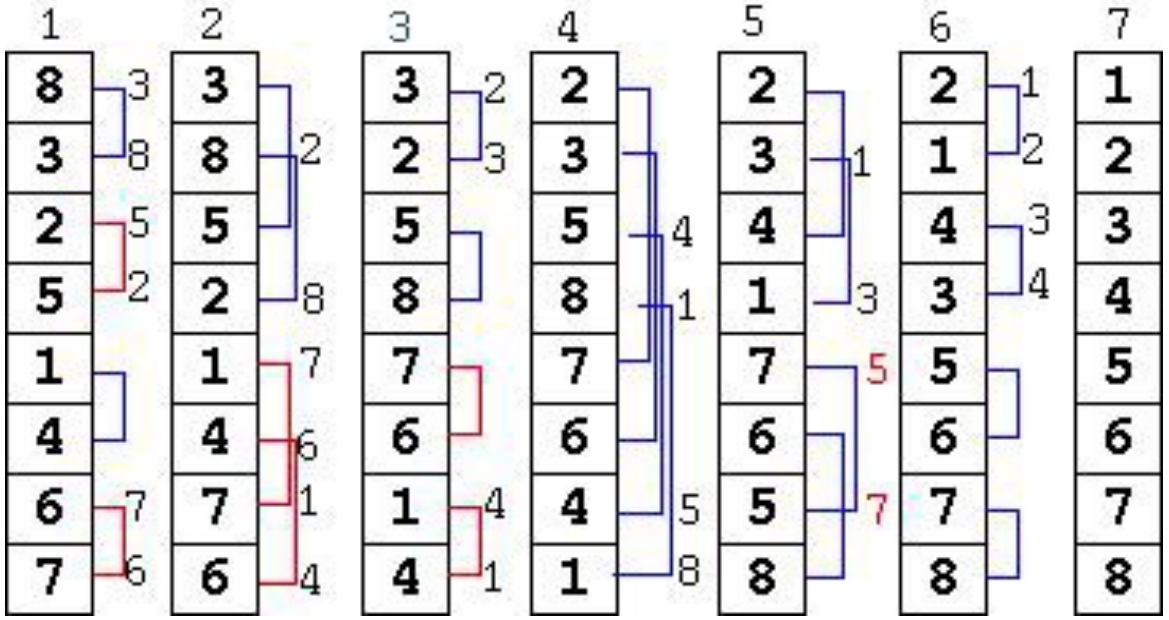


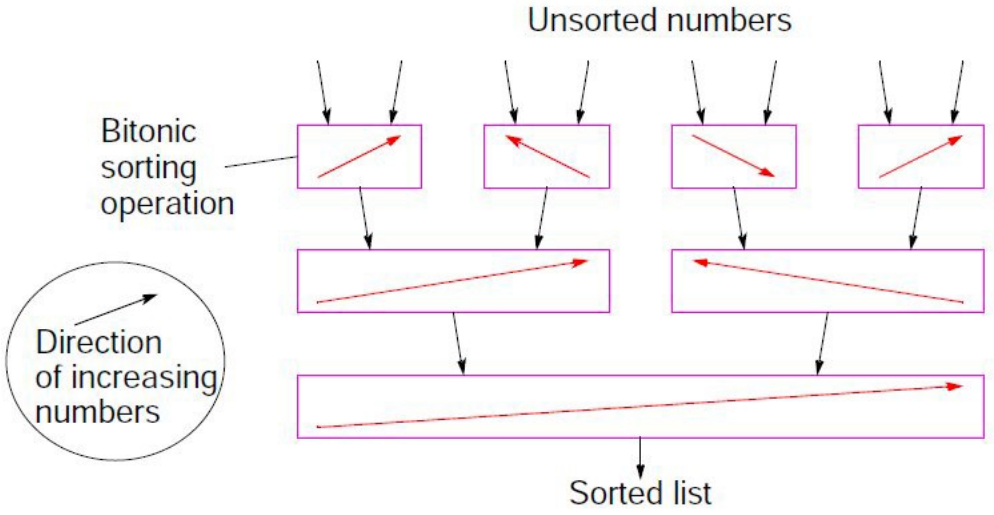
Parallel Sorting

Bitonic Merge Sort

- To sort an unsorted sequence, we first transform it in a bitonic sequence. Starting from adjacent pairs of values of the given unsorted sequence, bitonic sequences are created and then recursively merged into (twice the size) larger bitonic sequences. At the end, a single bitonic sequence is obtained, which is then sorted into the final increasing sequence.

Example





Parallel Sorting

Bitonic Merge Sort Example

Step 1: Convert the Sequence into a Bitonic Sequence (We divide the array into two halves):

- First half sorted in ascending order.
- Second half sorted in descending order.

Given array: [8,3,2,5,1,4,6,7]

Sort the first half in ascending order [2,3,5,8]

Sort the second half in descending order [7,6,4,1]

Now we have a **bitonic sequence**: [2,3,5,8,7,6,4,1]

Step 2: Bitonic Merge

Step 2.1: First Level of Comparisons: We compare the **first half** with the **second half**, swapping elements if necessary so that the first half contains smaller values and the second half contains larger values. (distance=4)

(2,7)⇒(2,7)(no swap)

(3,6)⇒(3,6)(no swap)

(5,4)⇒(4,5)(swap)

(8,1)⇒(1,8)(swap)

New sequence: **[2,3,4,1,7,6,5,8]**

Parallel Sorting

Bitonic Merge Sort Example

Step 2.2: Second Level of Comparisons:

Now, divide into groups of 2 and sort them(Compare 2 position apart, distance=2).

First half [2,3,4,1]:

(2,4)⇒(2,4)(no swap)

(3,1)⇒(1,3)(swap)

Updated first half:

[2,1,4,3]

Second half [7,6,5,8]:

(7,5)⇒(5,7)(swap)

(6,8)⇒(6,8)(no swap)

Updated second half:

[5,6,7,8]

New sequence:

[2,1,4,3,5,6,7,8]

Step 2.3: Third Level of Comparisons

Now, perform pairwise comparisons for a 1 step apart elements.

First half [2,1,4,3]:

(2,1)⇒(1,2)(swap)

(4,3)⇒(3,4)(swap)

Updated first half:

[1,2,3,4]

Second half [5,6,7,8]:

(5,6)⇒(5,6)(no swap)

(7,8)⇒(7,8)(no swap)

Updated second half:

[5,6,7,8]

Final Sorted Sequence

[1,2,3,4,5,6,7,8]

Parallel Sorting

Bitonic Merge Sort

The bitonic mergesort algorithm can be split into **k** phases (with $n = 2^k$):

Phase 1: convert adjacent pair of values into increasing/decreasing sequences, i.e., into *4-bit* bitonic sequences

Phases 2 to k-1: sort each *m-bit* bitonic sequence into increasing/decreasing sequences and merge adjacent sequences into a *2m-bit* bitonic sequence, until reaching a *n-bit* bitonic sequence

Phase k: sort the *n-bit* bitonic sequence

Given an unsorted list of size $n = 2^k$, there are k phases, with each phase i taking i parallel steps.

Therefore, in total it takes $\sum_{i=1}^{k} i = \frac{k+1}{2} = \frac{\log(n)+(\log(n)+1)}{2}$ parallel steps to obtain the final sorted list in a parallel implementation, which corresponds to a time complexity of $O(log^2 n)$.

Parallel Sorting

Time Complexity Summary

Algorithm	Sequential	Parallel
Bubble Sort	$O(n^2)$	$O(n)$ *
Merge Sort	$O(nlog(n))$	$O(n)$
Quick Sort	$O(nlog(n))$ **	$O(n)$ ***
Odd-Even Merge Sort	-	$O(log^2n)$ ****
Bitonic Sort	-	$O(log^2n)$

* : odd-even transposition sort
**: average time complexity
***: ideal pivot selection
****: recursive odd-even merge

External sorting and its applications in large-scale data engineering

Student presentation

References

1. **Sequential Sorting**: Cormen, T. H.; Leiserson, C. E.; Rivest, R. L. & Stein, C. (2001), Introduction to Algorithms , The MIT Press.
2. **Parallel Sorting Algorithms**: Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers. B. Wilkinson, M. Allen, Prentice Hall
3. Various resources Like books, Lecture slides from different universities, Web Links, AI tools etc.

*** End of Chapter 5 ***